

Dal Perceptron a ChatGpt

Come funziona una rete neurale, partendo da zero

Cristian Taccioli,

MAPS Dept. University of Padova - ITA version: Dept. of Computer Science and Technology, University of Cambridge, Thomas JJ Avenue, Cambridge, Aug 2026

Come si usa questa dispensa. Ogni volta che vedi il simbolo **A MANO** devi fermarti, prendere carta e penna e rifare tu il conto. Non serve la calcolatrice: tutti i numeri hanno una sola cifra dopo la virgola. Se non rifai i conti, non capisci davvero: è l'unica regola del corso. Le prime dodici pagine si possono seguire con la matematica del biennio. Dalla sezione sulle derivate in poi serve sapere che cosa è una pendenza, e quel poco che serve viene spiegato lì.

1. Il problema

Immagina di dover decidere ogni sabato sera se andare al cinema oppure restare a casa. La decisione dipende da due cose soltanto:

- **quanti amici vengono:** nessuno, uno, due, tre, quattro;
- **quanto è lontano il cinema:** la distanza da casa tua, in chilometri.

Sabato dopo sabato tu hai preso una decisione. Adesso vogliamo costruire una macchina che, guardando le tue decisioni passate, impari da sola la tua regola e sappia decidere al posto tuo anche in situazioni nuove che non ha mai visto.

Questa macchina si chiama **perceptron** ed è il mattone elementare di tutte le reti neurali, comprese quelle enormi che oggi traducono le lingue, riconoscono i volti o scrivono testi. È stata inventata nel 1958 da Frank Rosenblatt, uno psicologo americano, e la sua idea era semplicissima: copiare, in modo molto rozzo, quello che fa un neurone del cervello.

Come si trasforma una situazione in numeri

Un computer non capisce le parole, capisce solo le cifre. Quindi la prima cosa da fare è tradurre tutto in numeri. Questa operazione si chiama **codifica** ed è il primo passo di qualunque progetto di intelligenza artificiale.

Variabile	Significato	Valori
x_1	numero di amici che vengono	da 0 a 4
x_2	distanza di casa dal cinema	un numero di chilometri, da 1 a 7
y	la risposta vera	0 = resto a casa, 1 = vado al cinema

Le prime due variabili si chiamano **input** (o *caratteristiche*, in inglese *features*): sono le informazioni che entrano nella macchina. L'ultima, y , si chiama **etichetta** (in inglese *label*) oppure **outcome**: è la risposta giusta, quella che tu hai davvero dato quel sabato.

Due input soli non sono una limitazione: sono una scelta didattica. Con due input ogni sabato è un punto su un foglio a quadretti, e potremo disegnare tutto. Più avanti vedremo che cosa succede quando gli input diventano tre, dieci o diecimila.

2. Il dataset

Ecco i dieci sabati che useremo per insegnare al perceptron. Questo gruppo di dati si chiama **training set**, cioè insieme di addestramento.

riga	x_1 amici	x_2 km	y	cosa è successo
R1	1	1	1	un amico e il cinema è sotto casa: si va
R2	1	3	0	un amico solo non vale tre chilometri
R3	2	3	1	in due, tre chilometri si fanno
R4	0	3	0	da solo e lontano: resto a casa
R5	3	5	1	in tre si va anche a cinque chilometri
R6	2	5	0	in due, cinque chilometri sono troppi
R7	4	6	1	tutta la compagnia: sei chilometri si fanno
R8	3	7	0	in tre, sette chilometri sono troppi
R9	0	1	0	da solo non ci vado neanche se è vicino
R10	4	7	1	con quattro amici anche sette chilometri vanno bene

Il dataset è **bilanciato**: cinque volte siamo andati ($y = 1$) e cinque volte siamo rimasti a casa ($y = 0$). È importante, perché se avessimo avuto nove *no* e un solo *sì* la macchina potrebbe imparare la strategia stupida “rispondi sempre no” e sbagliare solo una volta su dieci.

Guardando la tabella si intuisce già la regola: *ogni amico in più mi rende disposto a fare un po' più di strada*. Ma noi non la diremo mai alla macchina. Il suo lavoro è tirarla fuori da sola.

3. Che cos'è un perceptron

Un perceptron è una macchinetta che fa tre cose in fila, sempre le stesse.

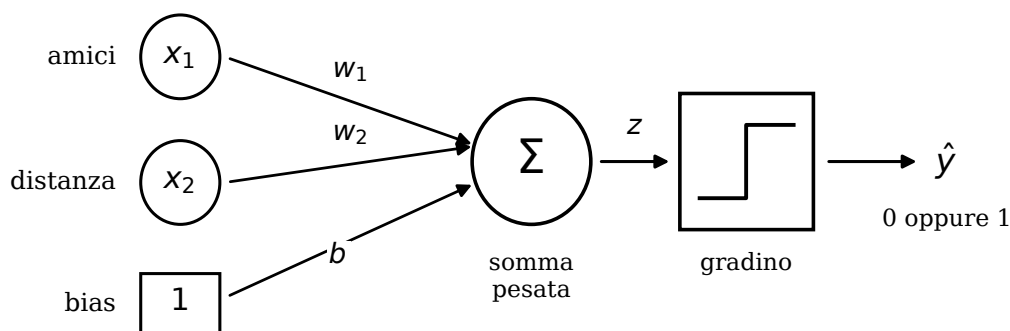


Figura 1: L'anatomia di un perceptron. Gli input entrano da sinistra, ognuno viene moltiplicato per il suo peso, i risultati si sommano insieme al bias e il totale passa nella funzione a gradino che decide 0 oppure 1.

3.1 Primo: dare un peso a ogni informazione

Non tutte le informazioni contano uguale. Il perceptron associa a ogni input x_i un numero w_i che si chiama **peso** (in inglese *weight*).

- Un peso **positivo e grande** vuol dire “questa cosa mi spinge ad andare”.
- Un peso **negativo** vuol dire “questa cosa mi frena”.
- Un peso **vicino a zero** vuol dire “questa cosa non mi interessa”.

Ti aspetteresti che il peso degli amici sia positivo e quello della distanza sia negativo. Vedremo che la macchina ci arriva da sola, ma non subito.

3.2 Secondo: fare la somma pesata e aggiungere il bias

Si moltiplica ogni input per il suo peso, si somma tutto e si aggiunge un numero fisso b chiamato **bias**:

$$z = w_1x_1 + w_2x_2 + b$$

Il numero z che esce da questo conto è il **punteggio** della serata.

3.3 Terzo: decidere con la funzione a gradino

Il punteggio z è un numero qualunque, ma la risposta deve essere solo sì o no. Allora si applica la **funzione a gradino** (in inglese *step function*):

$$\hat{y} = \begin{cases} 1 & \text{se } z \geq 0 \quad (\text{ANDIAMO}) \\ 0 & \text{se } z < 0 \quad (\text{RESTIAMO A CASA}) \end{cases}$$

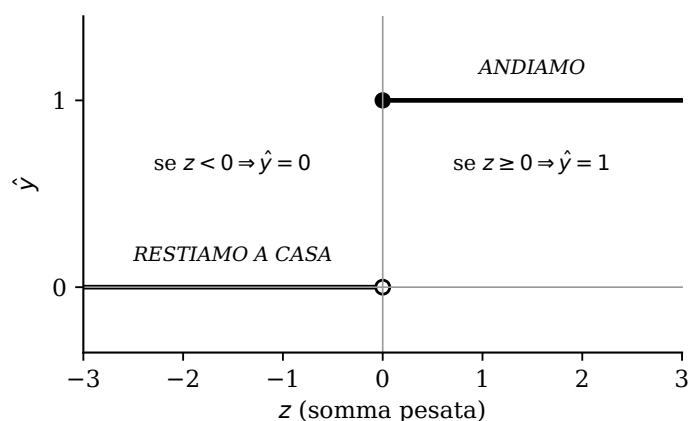


Figura 2: La funzione a gradino. Non c'è nulla di intermedio: o si sale sul gradino (risposta 1) o si resta giù (risposta 0). Il salto avviene esattamente in $z = 0$.

Perché si chiama così

Un neurone vero riceve segnali dagli altri neuroni attraverso le sinapsi. Alcune sinapsi sono forti, altre deboli: sono i pesi. Il neurone somma tutti i segnali che riceve e, se la somma supera una certa soglia, “spara” un impulso elettrico; se non la supera, resta zitto. Non spara a metà. Questo comportamento tutto o niente è esattamente la funzione a gradino. Il bias è la soglia, spostata dall'altra parte dell'uguale.

Il simbolo $\hat{y}$

Scriviamo y (si legge “epsilon”) per la risposta *vera*, quella scritta nel dataset, e $\hat{y}$ (si legge “epsilon cappello”) per la risposta *indovinata dalla macchina*. Quando $\hat{y} = y$ la macchina ha azzeccato, quando $\hat{y} \neq y$ ha sbagliato. Tutto l'addestramento serve a far diventare $\hat{y}$ uguale a y il più spesso possibile.

4. Il bias, capito bene

Il bias è il numero che gli studenti liquidano più in fretta e che invece nasconde l'idea più elegante del perceptron. Ci sono tre modi di guardarlo e conviene averli tutti e tre in testa.

Primo modo: il bias è la soglia, travestita

Riscriviamo la condizione “il perceptron dice sì”:

$$w_1x_1 + w_2x_2 + b \geq 0 \quad \Longleftrightarrow \quad w_1x_1 + w_2x_2 \geq -b$$

Letta così la formula dice: *fai la somma pesata delle informazioni e confrontala con la soglia $-b$* . Il bias non è altro che la soglia cambiata di segno e portata dall'altra parte. Un bias molto negativo significa una soglia alta, cioè un tipo difficile da convincere.

Secondo modo: il bias è la risposta quando non succede niente

Metti $x_1 = 0$ e $x_2 = 0$ nella formula: resta $z = b$. Quindi:

*il bias è il punteggio della serata in cui non c'è nessun amico
e il cinema è a zero chilometri, cioè la tua voglia di uscire “di base”.*

Se $b > 0$ sei un tipo che esce sempre e serve un buon motivo per farti restare a casa. Se $b < 0$ sei un casalingo e serve un buon motivo per farti uscire.

Terzo modo, il più importante: senza bias la frontiera passa per l'origine

Questo è il punto che decide tutto. Facciamo l'esempio più piccolo possibile: un solo input, il numero di amici, e la regola “vado al cinema se ci sono almeno due amici”.

A MANO Prova a realizzarla *senza bias*, cioè con $z = w_1x_1$ e basta. Con w_1 positivo:

$$z = w_1 \cdot 0 = 0 \geq 0 \Rightarrow \hat{y} = 1$$

Con zero amici la macchina dice già di sì, e con più amici z cresce ancora, quindi dice sempre di sì. Con w_1 negativo succede il contrario e dice sempre di no (tranne che per $x_1 = 0$). **Non esiste nessun w_1 che realizzi “almeno due amici”.**

A MANO Adesso rimetti il bias, con $w_1 = 1$ e $b = -2$:

$$z = x_1 - 2$$

x_1	$z = x_1 - 2$	$\hat{y}$	giusto?
0	-2	0	sì
1	-1	0	sì
2	0	1	sì
3	+1	1	sì

Funziona perfettamente. Lo stesso vale in due dimensioni: senza bias la retta di separazione è **obbligata a passare per l'origine** (0,0), perché $w_1 \cdot 0 + w_2 \cdot 0 = 0$ sempre. Il bias è ciò che le permette di staccarsi dall'origine e mettersi dove serve.

Il trucco dell'input sempre acceso

Guarda di nuovo la Figura 1: il bias è disegnato come un input che vale sempre 1, con il suo peso b . Non è un disegno di comodo, è un modo di vedere del tutto legittimo. Se poni $x_0 = 1$ e chiami $w_0 = b$, la formula diventa

$$z = w_0x_0 + w_1x_1 + w_2x_2$$

e il bias sparisce come concetto separato: è semplicemente il peso di un input finto che è sempre acceso. Questo trucco è comodissimo, perché così la regola di aggiornamento dei pesi vale anche per il bias senza scriverne una diversa. Nelle librerie vere il bias è trattato esattamente così.

Una nota di onestà

Nel nostro dataset del cinema, per puro caso, esisterebbe anche una retta attraverso l'origine capace di separare i dati (è la retta $x_2 = 2x_1$). Ma la macchina non lo sa in anticipo, e infatti troverà una retta diversa, che l'origine non la tocca. In generale il bias serve, e toglierlo è uno degli errori più comuni di chi comincia.

5. Il primo calcolo: la macchina appena nata

Ecco il punto più importante da capire di tutto il corso: **all'inizio la macchina non sa nulla**. Nessuno le ha detto che la distanza è una cosa negativa o che gli amici aiutano. I pesi e il bias vengono scelti **a caso**, come se lanciassimo dei dadi.

Supponiamo che il caso ci abbia dato questi valori di partenza:

w_1 (amici)	w_2 (distanza)	b (bias)
0,2	0,3	-0,1

Guarda bene: il caso ha messo $w_2 = +0,3$, cioè un peso *positivo* sulla distanza. Vuol dire che questa macchina appena nata crede che *più il cinema è lontano, più ho voglia di andarci*. È una sciocchezza, ma lei non lo sa ancora. Il nostro lavoro è farglielo scoprire dai dati.

Useremo anche un altro numero, il **tasso di apprendimento** η (si legge “eta”), che decide quanto grandi sono le correzioni. Noi useremo $\eta = 0,1$: correzioni piccole e prudenti.

Riga R1: $x = (1, 1)$, **risposta vera** $y = 1$

A MANO Sostituisci i numeri nella formula e fai la somma:

$$\begin{aligned} z &= w_1 x_1 + w_2 x_2 + b \\ &= 0,2 \cdot 1 + 0,3 \cdot 1 + (-0,1) \\ &= 0,2 + 0,3 - 0,1 = \mathbf{0,4} \end{aligned}$$

Applichiamo il gradino: $0,4 \geq 0$, quindi $\hat{y} = 1$. La risposta vera era $y = 1$. **Ha indovinato!** Quando la macchina indovina non si tocca nulla: i pesi restano come erano.

Riga R2: $x = (1, 3)$, **risposta vera** $y = 0$

A MANO Stessi pesi, dati diversi:

$$z = 0,2 \cdot 1 + 0,3 \cdot 3 + (-0,1) = 0,2 + 0,9 - 0,1 = \mathbf{1,0}$$

Il gradino dice: $1,0 \geq 0$, quindi $\hat{y} = 1$, cioè “andiamo”. Ma la risposta vera era $y = 0$: quel sabato eravamo rimasti a casa perché un amico solo non valeva tre chilometri. **Ha sbagliato**, e il colpevole si vede a occhio nudo: il termine $0,3 \cdot 3 = 0,9$ da solo vale più di tutto il resto. È il peso sbagliato della distanza.

Adesso dobbiamo correggerla.

6. La regola di apprendimento

Rosenblatt ha inventato una regola di correzione di una semplicità disarmante. Prima si calcola l'errore:

$$e = y - \hat{y}$$

Ci sono solo tre casi possibili, perché y e $\hat{y}$ valgono solo 0 o 1:

y	$\hat{y}$	$e = y - \hat{y}$	che cosa vuol dire
1	1	0	ha indovinato: non toccare niente
0	0	0	ha indovinato: non toccare niente
1	0	+1	doveva dire sì e ha detto no: alza il punteggio
0	1	-1	doveva dire no e ha detto sì: abbassa il punteggio

Poi si aggiorna ogni peso con questa formula:

$$w_i^{\text{nuovo}} = w_i^{\text{vecchio}} + \eta \cdot e \cdot x_i$$

$$b^{\text{nuovo}} = b^{\text{vecchio}} + \eta \cdot e$$

Perché questa formula funziona: leggila a voce alta

La formula dice: *correggi ogni peso in proporzione a quanto quell'input era acceso*. Guarda il fattore x_i alla fine.

Se $x_i = 0$, quell'input era spento, non ha contribuito al conto e quindi non ha colpa: $\eta \cdot e \cdot 0 = 0$, il peso non cambia. Giusto così.

Se x_i è grande (per esempio $x_2 = 6$ chilometri), quell'input ha spinto tantissimo il risultato e quindi ha molta colpa: viene corretto sei volte più di un input che valeva 1.

Il segno di e decide la direzione. Se $e = -1$ (abbiamo detto sì e non dovevamo) tutti i pesi degli input accesi scendono, così la prossima volta il punteggio sarà più basso. Se $e = +1$ salgono.

E il bias? Il bias si aggiorna con $\eta \cdot e$ e basta, senza moltiplicare per niente. Ma è la stessa formula: ricordi il trucco dell'input sempre acceso? Il bias ha $x_0 = 1$, quindi $\eta \cdot e \cdot 1 = \eta \cdot e$.

Correggiamo la riga R2

Abbiamo $y = 0$ e $\hat{y} = 1$, quindi $e = 0 - 1 = -1$. Gli input erano $x = (1, 3)$ e il tasso è $\eta = 0,1$.

A MANO Un numero alla volta:

$$w_1 = 0,2 + 0,1 \cdot (-1) \cdot 1 = 0,2 - 0,1 = \mathbf{0,1}$$

$$w_2 = 0,3 + 0,1 \cdot (-1) \cdot 3 = 0,3 - 0,3 = \mathbf{0,0}$$

$$b = -0,1 + 0,1 \cdot (-1) = \mathbf{-0,2}$$

Guarda che cosa è successo a w_2 : era 0,3 ed è crollato a 0,0 in un colpo solo, perché la distanza valeva 3 e quindi si è presa tre volte la correzione degli amici. La macchina ha appena imparato la sua prima lezione sul mondo: *la distanza non conta in positivo*.

Riga R3: $x = (2, 3)$, **risposta vera** $y = 1$

Attenzione: adesso i pesi sono quelli *nuovi*, cioè $w = (0,1; 0,0)$ e $b = -0,2$. Il perceptron impara riga per riga, non aspetta la fine.

A MANO

$$z = 0,1 \cdot 2 + 0,0 \cdot 3 + (-0,2) = 0,2 + 0 - 0,2 = \mathbf{0,0}$$

$0,0 \geq 0$, quindi $\hat{y} = 1$: giusto, era proprio $y = 1$. Nessuna correzione. Nota che qui la macchina ha indovinato per un pelo, sedendosi esattamente sul bordo.

Righe R4 e R5

A MANO R4 è $x = (0, 3)$ con $y = 0$:

$$z = 0,1 \cdot 0 + 0,0 \cdot 3 - 0,2 = \mathbf{-0,2} \Rightarrow \hat{y} = 0 \text{ giusto}$$

R5 è $x = (3, 5)$ con $y = 1$:

$$z = 0,1 \cdot 3 + 0,0 \cdot 5 - 0,2 = 0,3 - 0,2 = \mathbf{+0,1} \Rightarrow \hat{y} = 1 \text{ giusto}$$

Riga R6: il momento in cui la macchina capisce

A MANO R6 è $x = (2, 5)$ con $y = 0$:

$$z = 0,1 \cdot 2 + 0,0 \cdot 5 - 0,2 = 0,2 - 0,2 = \mathbf{0,0} \Rightarrow \hat{y} = 1$$

Ma $y = 0$: sbagliato, $e = -1$. Correggiamo:

$$w_1 = 0,1 + 0,1 \cdot (-1) \cdot 2 = 0,1 - 0,2 = -0,1$$

$$w_2 = 0,0 + 0,1 \cdot (-1) \cdot 5 = 0,0 - 0,5 = -0,5$$

$$b = -0,2 - 0,1 = -0,3$$

Il peso della distanza è diventato negativo. Da adesso ogni chilometro in più abbassa il punteggio. La macchina ha capito da sola, solo guardando i dati, che il cinema lontano scoraggia. Nessuno gliel'ha detto.

Ha però esagerato: adesso anche il peso degli amici è negativo, il che è assurdo. Le righe successive glielo faranno correggere.

7. L'epoca 1 per intero

Un passaggio completo su tutte e dieci le righe si chiama **epoca** (in inglese *epoch*). Ecco l'epoca 1 al completo. Le prime sei righe le hai già fatte tu: controlla che coincidano, poi continua da R7 fino a R10.

riga	input			prima						dopo		
	x_1	x_2	y	w_1	w_2	b	z	$\hat{y}$	e	w_1	w_2	b
R1	1	1	1	0,2	0,3	-0,1	+0,4	1	0	nessun cambiamento		
R2	1	3	0	0,2	0,3	-0,1	+1,0	1	-1	0,1	0,0	-0,2
R3	2	3	1	0,1	0,0	-0,2	0,0	1	0	nessun cambiamento		
R4	0	3	0	0,1	0,0	-0,2	-0,2	0	0	nessun cambiamento		
R5	3	5	1	0,1	0,0	-0,2	+0,1	1	0	nessun cambiamento		
R6	2	5	0	0,1	0,0	-0,2	0,0	1	-1	-0,1	-0,5	-0,3
R7	4	6	1	-0,1	-0,5	-0,3	-3,7	0	+1	0,3	0,1	-0,2
R8	3	7	0	0,3	0,1	-0,2	+1,4	1	-1	0,0	-0,6	-0,3
R9	0	1	0	0,0	-0,6	-0,3	-0,9	0	0	nessun cambiamento		
R10	4	7	1	0,0	-0,6	-0,3	-4,5	0	+1	0,4	0,1	-0,2

Bilancio dell'epoca 1: 5 errori su 10 righe. Pesi alla fine: $w = (0,4; 0,1)$ e $b = -0,2$.

Nota il saliscendi del peso della distanza: $0,3 \rightarrow 0,0 \rightarrow -0,5 \rightarrow +0,1 \rightarrow -0,6 \rightarrow +0,1$. L'apprendimento non è una linea dritta: ogni riga tira i pesi dalla sua parte e ci vuole un po' prima che si trovi un compromesso che vada bene per tutti e dieci i sabati.

Epoca 2

Si ricomincia da capo dalla riga R1, ma con i pesi arrivati alla fine dell'epoca 1.

riga	input			prima						dopo		
	x_1	x_2	y	w_1	w_2	b	z	$\hat{y}$	e	w_1	w_2	b
R1	1	1	1	0,4	0,1	-0,2	+0,3	1	0	nessun cambiamento		
R2	1	3	0	0,4	0,1	-0,2	+0,5	1	-1	0,3	-0,2	-0,3
R3	2	3	1	0,3	-0,2	-0,3	-0,3	0	+1	0,5	0,1	-0,2
R4	0	3	0	0,5	0,1	-0,2	+0,1	1	-1	0,5	-0,2	-0,3
R5	3	5	1	0,5	-0,2	-0,3	+0,2	1	0	nessun cambiamento		
R6	2	5	0	0,5	-0,2	-0,3	-0,3	0	0	nessun cambiamento		
R7	4	6	1	0,5	-0,2	-0,3	+0,5	1	0	nessun cambiamento		
R8	3	7	0	0,5	-0,2	-0,3	-0,2	0	0	nessun cambiamento		
R9	0	1	0	0,5	-0,2	-0,3	-0,5	0	0	nessun cambiamento		
R10	4	7	1	0,5	-0,2	-0,3	+0,3	1	0	nessun cambiamento		

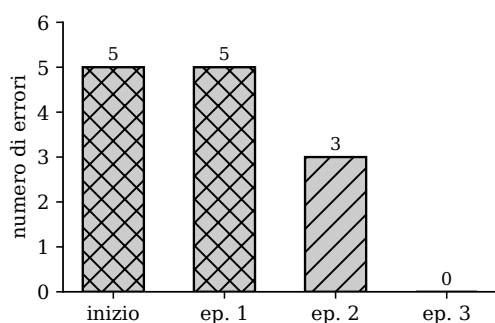
Bilancio dell'epoca 2: 3 errori, tutti nelle prime quattro righe. Da R5 in poi non sbaglia più niente. Pesi finali: $w = (0,5; -0,2)$, $b = -0,3$.

Epoca 3: la macchina ha imparato

Nell'epoca 3 succede una cosa bellissima: la macchina passa su tutte e dieci le righe e non sbaglia mai. Verificalo tu, riga per riga, con $w = (0,5; -0,2)$ e $b = -0,3$.

riga	x_1	x_2	y	il conto	z	$\hat{y}$
R1	1	1	1	$0,5 - 0,2 - 0,3$	0,0	1
R2	1	3	0	$0,5 - 0,6 - 0,3$	-0,4	0
R3	2	3	1	$1,0 - 0,6 - 0,3$	+0,1	1
R4	0	3	0	$0,0 - 0,6 - 0,3$	-0,9	0
R5	3	5	1	$1,5 - 1,0 - 0,3$	+0,2	1
R6	2	5	0	$1,0 - 1,0 - 0,3$	-0,3	0
R7	4	6	1	$2,0 - 1,2 - 0,3$	+0,5	1
R8	3	7	0	$1,5 - 1,4 - 0,3$	-0,2	0
R9	0	1	0	$0,0 - 0,2 - 0,3$	-0,5	0
R10	4	7	1	$2,0 - 1,4 - 0,3$	+0,3	1

Dieci righe su dieci corrette. Nessun errore vuol dire nessuna correzione, e nessuna correzione vuol dire che i pesi non cambieranno mai più: l'addestramento è finito. Si dice che l'algoritmo è **andato a convergenza**.



La curva di apprendimento. Gli errori scendono da 5 a 3 a 0. In tre passaggi sui dati la macchina è passata dal non capire niente al capire tutto. Un grafico come questo si chiama *curva di apprendimento* e in ogni progetto di rete neurale è la prima cosa che si guarda: se non scende, qualcosa non va.

Che regola ha imparato?

I pesi finali sono $w_1 = 0,5$, $w_2 = -0,2$, $b = -0,3$. Adesso possiamo leggerli come si legge una frase:

- ogni **amico** aggiunge 0,5 al punteggio;
- ogni **chilometro** toglie 0,2: il segno è diventato negativo, come doveva essere;
- il **bias** è negativo (-0,3): siamo tipi un po' pigri, se non c'è nessun motivo particolare restiamo a casa.

Facciamo il cambio: quanti chilometri vale un amico? Basta chiedersi quanti chilometri servono per annullare 0,5 di punteggio, cioè $0,5 \div 0,2 = 2,5$. Quindi la macchina ha concluso che **ogni amico vale due chilometri e mezzo di strada**. Nessuno ha mai scritto questa regola: lei l'ha estratta da sola da dieci esempi. Questa è, in miniatura, tutta il sistema delle reti neurali.

8. La retta: da dove viene e come si muove

Questa è la sezione centrale della dispensa. Finora abbiamo fatto conti; adesso vediamoli ad occhio e capiamo che cosa quei conti stanno disegnando.

8.1 Perché la frontiera è proprio una retta

Il perceptron cambia idea in un punto solo: quando z passa da negativo a positivo. Il confine fra le due risposte è dunque l'insieme dei punti in cui z vale esattamente zero:

$$w_1 x_1 + w_2 x_2 + b = 0$$

Adesso guarda bene questa scrittura. Le incognite x_1 e x_2 compaiono solo alla prima potenza, moltiplicate per delle costanti, e sommate. È l'equazione di primo grado in due incognite che hai studiato alle superiori: la sua soluzione è una **retta**. Non è una scelta di chi ha progettato il perceptron, è una conseguenza inevitabile del fatto che il perceptron sa fare solo moltiplicazioni e somme.

8.2 Come si porta in forma esplicita

L'equazione $w_1 x_1 + w_2 x_2 + b = 0$ è nella cosiddetta *forma implicita*. Per disegnarla conviene isolare x_2 , cioè metterla nella forma $x_2 = m x_1 + q$ che sai già trattare. Risolvere per x_1 non cambia nulla, è la stessa cosa.

A MANO Segui i tre passaggi con i nostri pesi finali $w_1 = 0,5$, $w_2 = -0,2$, $b = -0,3$:

$0,5 x_1 - 0,2 x_2 - 0,3 = 0$	equazione di partenza
$-0,2 x_2 = -0,5 x_1 + 0,3$	porto a destra i termini senza x_2
$x_2 = \frac{-0,5 x_1 + 0,3}{-0,2}$	divido tutto per $-0,2$
$x_2 = 2,5 x_1 - 1,5$	semplifico

In generale, ripetendo gli stessi passaggi con le lettere:

$$x_2 = \underbrace{\left(-\frac{w_1}{w_2}\right)}_{\text{pendenza } m} x_1 + \underbrace{\left(-\frac{b}{w_2}\right)}_{\text{intercetta } q}$$

A MANO Verifica: $-\frac{w_1}{w_2} = -\frac{0,5}{-0,2} = +2,5$ e $-\frac{b}{w_2} = -\frac{-0,3}{-0,2} = -1,5$. Tornano.

8.3 Che cosa vogliono dire pendenza e intercetta

- La **pendenza** $m = 2,5$ dice di quanto sale la retta quando ti sposti di 1 verso destra. Nel nostro problema l'asse orizzontale sono gli amici e quello verticale i chilometri, quindi $m = 2,5$ significa esattamente *ogni amico in più mi fa accettare 2,5 chilometri in più di strada*. È lo stesso numero che avevamo trovato dividendo 0,5 per 0,2, e non è un caso: è la stessa divisione.
- L'**intercetta** $q = -1,5$ è il punto in cui la retta taglia l'asse verticale, cioè dove starebbe la soglia con zero amici. È negativa, il che vuol dire che con zero amici nessuna distanza è accettabile, neanche zero chilometri. Coerente con la riga R9, dove da solo siamo rimasti a casa anche a un chilometro.

8.4 Da che parte sta un punto: il test che decide tutto

Data la retta, come faccio a sapere se un punto sta nella zona “andiamo”? Non serve nessuna geometria: **basta sostituire le coordinate del punto nella formula di z e guardare il segno.**

A MANO Prendiamo il punto R7, cioè quattro amici e sei chilometri:

$$z = 0,5 \cdot 4 - 0,2 \cdot 6 - 0,3 = 2,0 - 1,2 - 0,3 = +0,5 > 0$$

Segno positivo: sta nella zona “andiamo”. E infatti $y = 1$.

Prendiamo R8, tre amici e sette chilometri:

$$z = 1,5 - 1,4 - 0,3 = -0,2 < 0$$

Segno negativo: zona “restiamo”. E infatti $y = 0$.

C'è però una domanda che va posta: la zona “andiamo” è sopra o sotto la retta? La risposta dipende dal segno di w_2 , ed è un punto che confonde quasi tutti. Ripercorriamo il passaggio algebrico:

$$w_1x_1 + w_2x_2 + b \geq 0 \quad \Rightarrow \quad w_2x_2 \geq -w_1x_1 - b$$

Adesso per isolare x_2 devo dividere per w_2 , e nelle disequazioni **dividere per un numero negativo capovolge il verso**:

$$\text{se } w_2 > 0 : \quad x_2 \geq mx_1 + q \quad (\text{“andiamo” è SOPRA})$$

$$\text{se } w_2 < 0 : \quad x_2 \leq mx_1 + q \quad (\text{“andiamo” è SOTTO})$$

Nel nostro caso $w_2 = -0,2 < 0$, quindi la zona “andiamo” è *sotto* la retta. Ha senso: sotto vuol dire poca distanza.

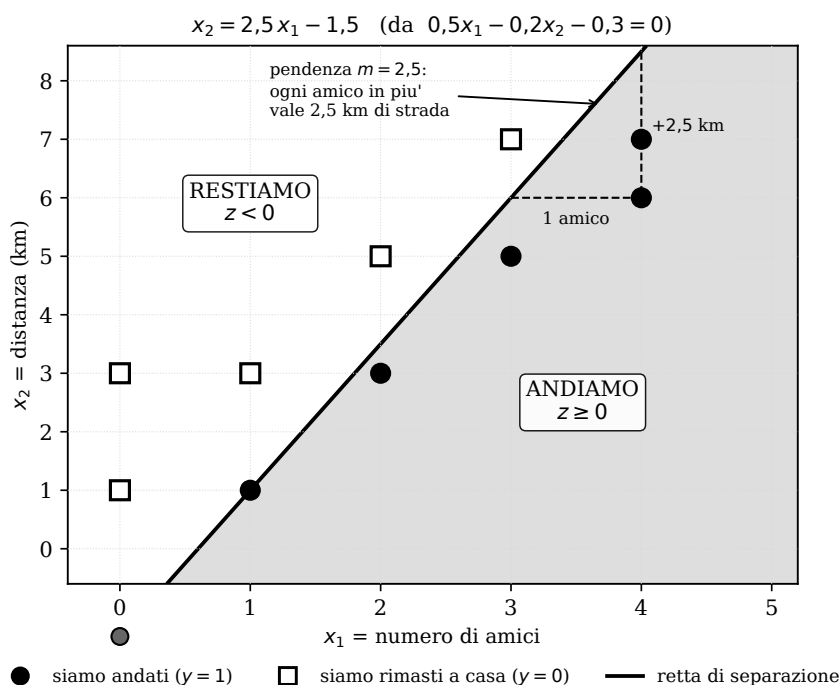


Figura 3: La retta imparata dal perceptron, con tutti i suoi pezzi. La zona grigia è dove $z \geq 0$, cioè dove la macchina risponde “andiamo”. Tutti i cerchi neri ci stanno dentro, tutti i quadrati bianchi ne stanno fuori.

Il caso limite: $w_2 = 0$

Se w_2 vale esattamente zero non puoi dividere e la formula esplicita non esiste. Che cosa succede geometricamente? L'equazione diventa $w_1x_1 + b = 0$, cioè $x_1 = -b/w_1$: una **retta verticale**. È perfettamente legittima, semplicemente non si può scrivere nella forma $x_2 = mx_1 + q$, esattamente come nelle rette che hai studiato a scuola. Nella tabella dell'epoca 1, dopo la riga R2, i pesi sono $w = (0,1; 0,0)$: in quel preciso momento la frontiera del nostro perceptron era la retta verticale $x_1 = 2$, cioè la macchina stava decidendo solo in base al numero di amici, ignorando del tutto la distanza.

8.5 Che cosa fa ciascuno dei tre numeri

Adesso il punto che rende tutto chiaro: i tre numeri w_1 , w_2 , b non fanno la stessa cosa alla retta.

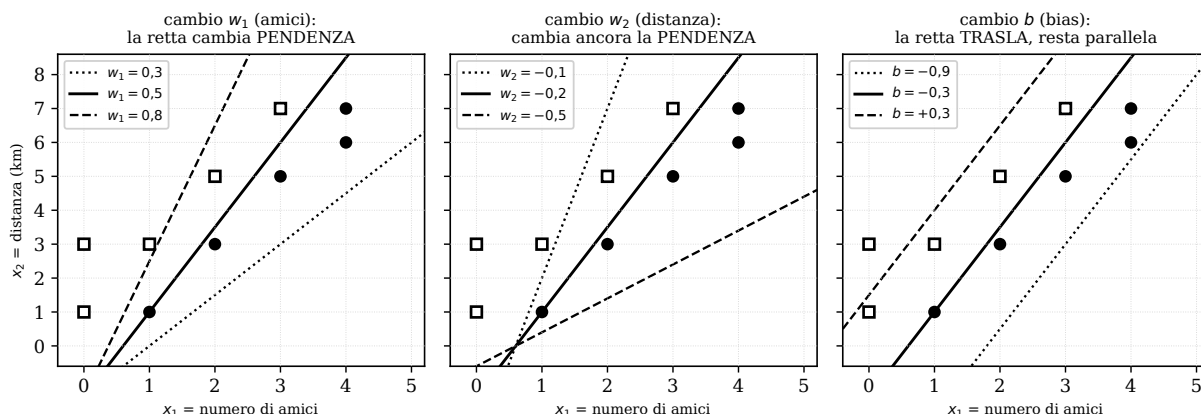


Figura 4: Il ruolo dei tre numeri. Cambiando w_1 oppure w_2 la retta **ruota**, perché cambia il rapporto $-w_1/w_2$ che è la pendenza. Cambiando il bias b la retta **trasla** rimanendo parallela a se stessa, perché b compare solo nell'intercetta $-b/w_2$.

se cambio	la retta	perché
w_1 (amici)	ruota	compare solo nella pendenza $-w_1/w_2$
w_2 (distanza)	ruota, e se cambia segno si ribalta anche il lato "andiamo"	compare sia nella pendenza sia nell'intercetta
b (bias)	trasla, resta parallela	compare solo nell'intercetta $-b/w_2$

A MANO Verifica il caso del bias. Con $w = (0,5; -0,2)$ e $b = -0,9$ l'intercetta diventa $q = -(-0,9)/(-0,2) = -4,5$, mentre la pendenza resta 2,5. La retta è la stessa, spostata in basso di 3. Un bias più negativo significa infatti un tipo più pigro, quindi una zona "andiamo" più piccola.

8.6 Come si muove la retta epoca dopo epoca

Adesso possiamo rileggere tutto l'addestramento come un film: a ogni correzione dei pesi la retta si sposta.

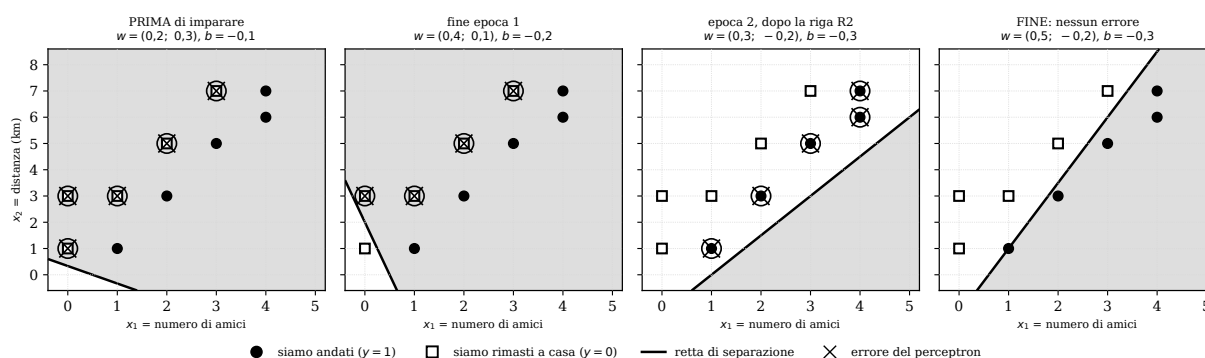


Figura 5: La retta durante l'addestramento. **Primo pannello**, i pesi casuali: w_2 è positivo, quindi la zona "andiamo" è sopra e copre quasi tutto il disegno; la macchina dice sì a tutti e sbaglia i cinque sabati in cui eravamo rimasti a casa. **Secondo**, fine dell'epoca 1: la retta si è raddrizzata ma w_2 è ancora positivo, quindi il lato sbagliato è ancora sopra. **Terzo**, epoca 2 subito dopo la riga R2: w_2 è appena diventato negativo e la zona grigia si ribalta di sotto; la macchina ha esagerato nella correzione e adesso dice no a tutti. **Quarto**, poche righe dopo si assesta: zero errori.

Ci sono due lezioni in questa figura, entrambe importanti.

La prima è che **l'apprendimento non è monotono**. Al terzo pannello la macchina sta momentaneamente peggio che al secondo. Il perceptron corregge sempre guardando una riga sola per volta e non ha nessuna idea di come sta andando sull'insieme; è come sistemare un quadro storto guardandolo solo da un angolo della stanza. Il teorema di Rosenblatt garantisce che alla fine ci arriva, non che migliori a ogni passo.

La seconda è che **il segno di w_2 decide da che parte sta la zona “andiamo”**. Il momento in cui w_2 passa da positivo a negativo è il momento in cui la macchina capisce davvero il problema, e sul disegno si vede come un ribaltamento del grigio.

8.7 Un altro modo di guardare la stessa cosa

C'è una visualizzazione ancora più diretta: invece dei punti nel piano, disegniamo su una sola riga i dieci valori del punteggio z calcolati con i pesi finali.

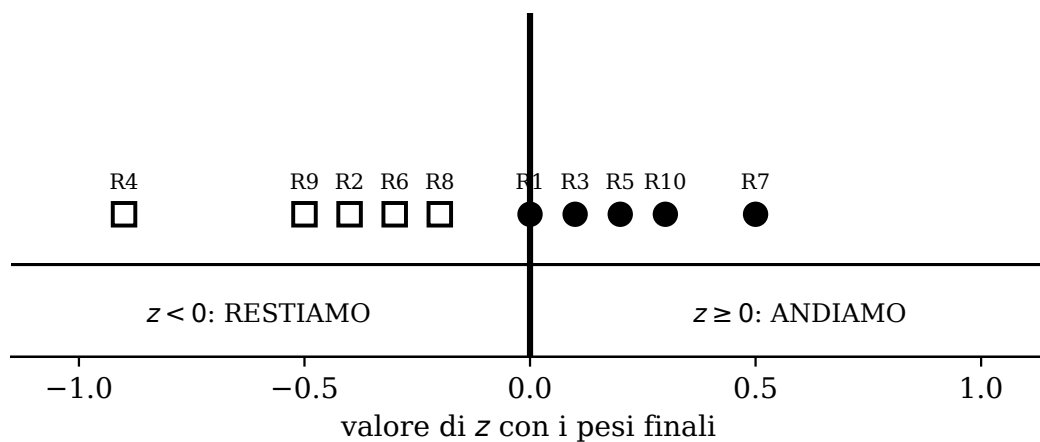


Figura 6: I dieci sabati ordinati per punteggio. La riga verticale spessa è la soglia $z = 0$. Tutti i cerchi neri stanno a destra o esattamente sulla soglia, tutti i quadrati bianchi stanno a sinistra.

Questo è, in una frase, tutto ciò che fa un perceptron: **schiaccia una situazione complicata in un solo numero e poi guarda se quel numero è positivo o negativo**. Tutto il resto della dispensa consisterà nel rendere più furbo questo schiacciamento.

9. E se gli input sono più di due?

La retta è una comodità del caso a due input. Vediamo che cosa succede in generale, perché serve fra poco.

input	equazione di $z = 0$	la frontiera è	si può disegnare?
1	$w_1x_1 + b = 0$	un punto sulla retta dei numeri	sì
2	$w_1x_1 + w_2x_2 + b = 0$	una retta nel piano	sì
3	$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$	un piano nello spazio	a fatica
n	$w_1x_1 + \dots + w_nx_n + b = 0$	un <i>iperpiano</i>	no, ma i conti sono identici

La parola *iperpiano* spaventa, ma non c'è nulla di misterioso: è solo il nome che si dà alla stessa equazione quando le lettere sono tante. Il conto da fare resta “moltiplica, somma, guarda il segno”, identico a quello che hai fatto a mano nelle pagine precedenti. Il fatto che non lo puoi disegnare è un problema tuo, non del perceptron.

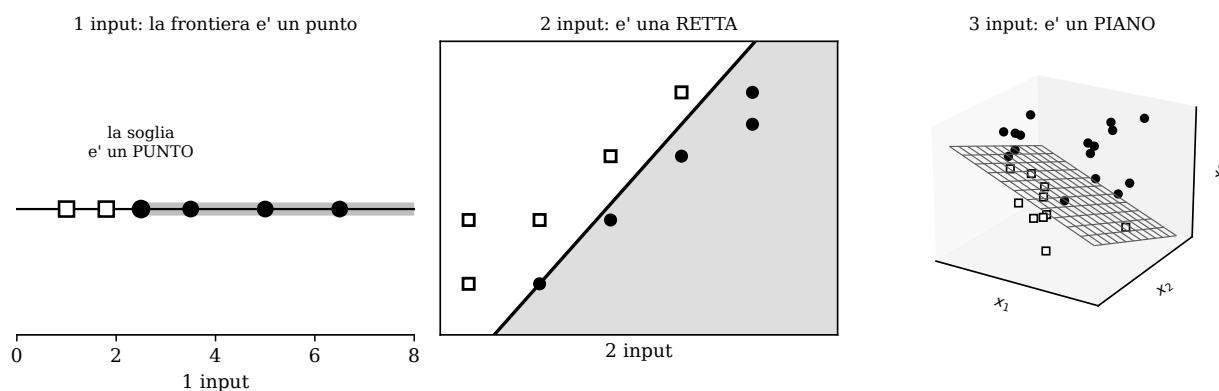


Figura 7: La stessa formula, in dimensioni diverse. La frontiera è sempre “l’oggetto piatto di una dimensione in meno rispetto allo spazio in cui vivi”.

La cosa da portarsi via

Un perceptron, qualunque sia il numero di input, sa fare una cosa sola: tagliare lo spazio in due con un taglio dritto. Tutto quello che verrà dopo nella dispensa, gli strati nascosti, la backpropagation, il deep learning, serve a superare questo limite.

10. Il momento della verità: validation e test

Abbiamo verificato che la macchina risponde bene sui dieci sabati che le abbiamo fatto studiare, ma quello è come dare a uno studente un compito con le stesse identiche domande che ha visto il giorno prima. Non dimostra niente: potrebbe averle imparate a memoria.

Per questo i dati si dividono sempre in **tre gruppi separati**.

gruppo	a che serve	la metafora della scuola
training	la macchina ci impara sopra, cambiando i pesi	gli esercizi svolti in classe con il professore
validation	si controlla se funziona anche fuori, e si decidono i dettagli (quante epoche, quanto vale η)	la simulazione della verifica, si può ripetere
test	si usa una volta sola , alla fine, e non si tocca più nulla	l'esame di maturità

La regola d'oro è: **sui dati di test non si allena mai**. Se sbirci il test e poi cambi qualcosa, il test è bruciato e non ti dice più la verità.

Validation: tre sabati mai visti

A MANO Calcola tu i tre punteggi con $w = (0,5; -0,2)$ e $b = -0,3$ prima di leggere la colonna z . I pesi **non si toccano più**.

	x_1	x_2	il conto	z	$\hat{y}$	vero y	esito
V1	2	2	$1,0 - 0,4 - 0,3$	+0,3	1	1	corretto
V2	1	4	$0,5 - 0,8 - 0,3$	-0,6	0	0	corretto
V3	3	4	$1,5 - 0,8 - 0,3$	+0,4	1	1	corretto

Tre su tre, cioè **accuratezza** 100%. L'accuratezza è semplicemente il numero di risposte giuste diviso il numero di domande.

Nel training non c'era nessun sabato uguale a questi. Rispondere bene lo stesso si chiama **generalizzare**: saper affrontare una domanda nuova mettendo insieme quello che si è capito da altre domande. È l'unica cosa che conta davvero nel machine learning.

Test: l'esame finale

A MANO Ultimi quattro sabati, quelli che decidono il voto.

	x_1	x_2	il conto	z	$\hat{y}$	vero y	esito
T1	4	3	$2,0 - 0,6 - 0,3$	+1,1	1	1	corretto
T2	0	5	$0,0 - 1,0 - 0,3$	-1,3	0	0	corretto
T3	2	6	$1,0 - 1,2 - 0,3$	-0,5	0	0	corretto
T4	3	2	$1,5 - 0,4 - 0,3$	+0,8	1	1	corretto

Quattro su quattro. Il perceptron ha superato l'esame. E' anche possibile far decidere alla macchina senza dargli l'esito... anzi è questo il motivo per cui si usa. Gli do delle cose da imparare nel training verifico che sia capace nel validation e poi lo uso per classificare e predire nella realtà.

Attenzione: 100% non è sempre una buona notizia

Qui abbiamo pochi dati, molto ordinati, costruiti apposta perché la regola esista davvero. Nella realtà le cose sono più sporche e un 100% perfetto è spesso il segnale che qualcosa è andato storto: di solito che i dati di test sono finiti per sbaglio anche nel training. Se un tuo modello fa 100%, la prima reazione deve essere il sospetto, non la festa.

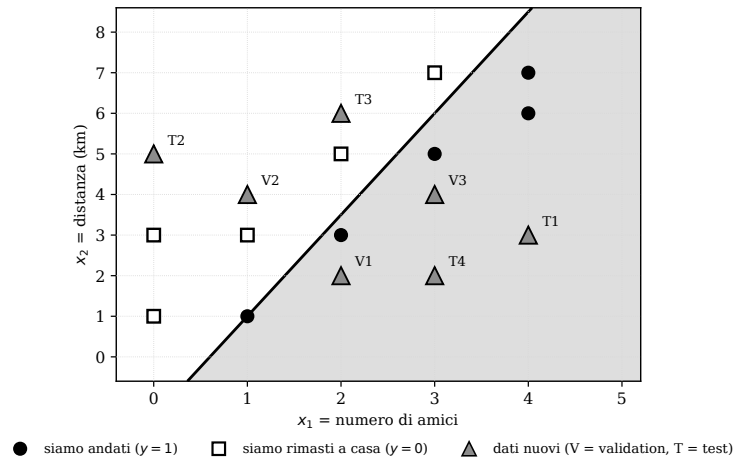


Figura 8: I sette dati nuovi (triangoli grigi) messi sopra la retta imparata dai soli dati di training. Nessuno ha spostato la retta per farli entrare: la retta era già lì, e loro sono capitati dalla parte giusta da soli.

11. Più neuroni nello stesso strato

Un solo perceptron traccia una retta sola. Ma se ne mettiamo due in parallelo, ciascuno con i suoi pesi e il suo bias, otteniamo **due rette** e possiamo combinarle.

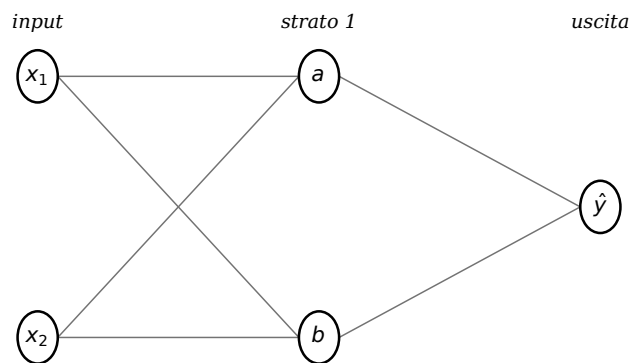


Figura 9: Due neuroni nello stesso strato. Gli stessi input arrivano a entrambi, ma i pesi sui collegamenti sono diversi, quindi le due risposte sono diverse.

Cambiamo di poco il problema del cinema, così serve davvero. La nuova regola di casa è:

*andiamo al cinema se ne vale la pena rispetto al numero di amici,
ma se il cinema è a meno di due chilometri ci andiamo a piedi
e ci fermiamo al bar, quindi al cinema non ci arriviamo.*

Servono due condizioni contemporaneamente e nessuna retta sola le può esprimere. Mettiamo allora due neuroni:

	w_1	w_2	b	che cosa controlla
neurone a	0,5	-0,2	-0,3	“ne vale la pena” (il perceptron che abbiamo addestrato)
neurone b	0,0	1,0	-2,0	“è abbastanza lontano da prendere l'autobus”

Il neurone b ha $w_1 = 0$: ignora completamente gli amici e guarda solo la distanza. Il suo punteggio è $z_b = x_2 - 2$, quindi si accende quando $x_2 \geq 2$.

A MANO Compila la tabella. Per ogni riga calcola prima z_a e z_b , poi applica il gradino a ciascuno.

x_1	x_2	$z_a = 0,5x_1 - 0,2x_2 - 0,3$	a	$z_b = x_2 - 2$	b	
1	1	+0,0	1	-1	0	troppo vicino
2	3	+0,1	1	+1	1	si va
0	3	-0,9	0	+1	1	nessun amico
3	5	+0,2	1	+3	1	si va
2	5	-0,3	0	+3	1	pochi amici per 5 km
4	6	+0,5	1	+4	1	si va
1	4	-0,6	0	+2	1	un amico non basta
4	3	+1,1	1	+1	1	si va

I due numeri di partenza sono stati **riscritti** in due numeri nuovi, la coppia (a, b) , che descrivono la serata in un linguaggio diverso. Questo si chiama **rappresentazione** ed è la cosa più importante che fa una rete neurale: non tanto decidere, quanto *riscrivere il problema in una forma in cui decidere diventa facile*.

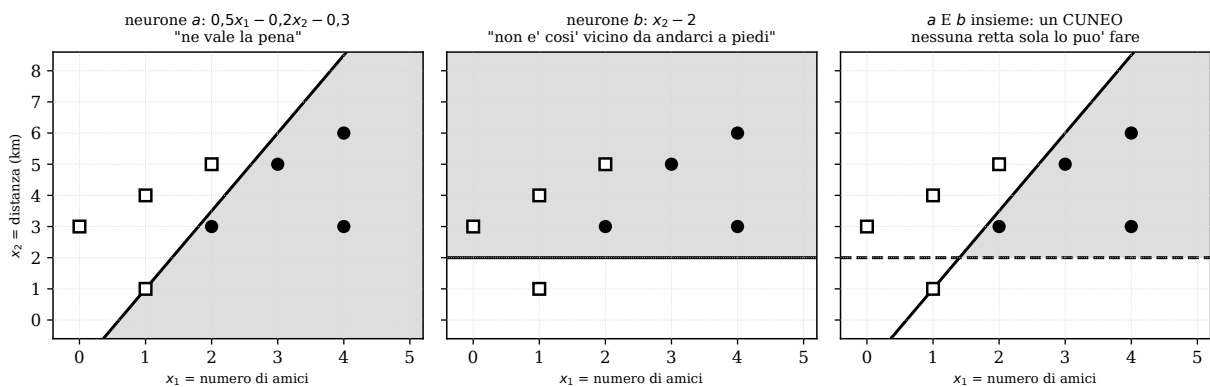


Figura 10: Due neuroni, due rette. Ciascuno da solo taglia il piano in due (pannelli di sinistra e centro). Ma se pretendiamo che si accendano *entrambi*, la zona buona è l'intersezione delle due, cioè un **cuneo** (pannello di destra). Un cuneo non è un semipiano e nessun perceptron singolo lo può disegnare.

12. Il secondo strato: che cosa cambia davvero

Adesso i due numeri a e b diventano gli input di un terzo neurone, che darà la risposta finale. Il calcolo è identico a prima, cambiano solo i nomi:

$$z_{\text{out}} = v_1 \cdot a + v_2 \cdot b + c \quad \text{e poi} \quad \hat{y} = \text{gradino}(z_{\text{out}})$$

dove v_1, v_2 sono i pesi del secondo strato e c è il suo bias. La regola da ricordare è: **l'uscita di uno strato diventa l'ingresso dello strato successivo**. Punto. Non c'è nient'altro da sapere.

A MANO Proviamo con $v_1 = 1, v_2 = 1, c = -1,5$. Per il sabato (2, 3), dove avevamo trovato $a = 1$ e $b = 1$:

$$z_{\text{out}} = 1 \cdot 1 + 1 \cdot 1 - 1,5 = +0,5 \Rightarrow \hat{y} = 1$$

E per il sabato (1, 1), dove $a = 1$ e $b = 0$:

$$z_{\text{out}} = 1 \cdot 1 + 1 \cdot 0 - 1,5 = -0,5 \Rightarrow \hat{y} = 0$$

Con questi pesi il neurone di uscita risponde 1 solo se *entrambi* i neuroni sotto si sono accesi: ha realizzato la **E logica** (in inglese AND), ed è proprio questa E che ritaglia il cuneo della figura precedente.

A MANO Adesso cambia solo il bias, da $-1,5$ a $-0,5$, e rifai i due conti. Otterrai $+1,5$ e $+0,5$, quindi 1 in entrambi i casi: adesso basta che *almeno uno* dei due si accenda. È la **O logica** (OR).

<i>a</i>	<i>b</i>	<i>z</i> con <i>c</i> = −1,5	AND	<i>z</i> con <i>c</i> = −0,5	OR
0	0	−1,5	0	−0,5	0
0	1	−0,5	0	+0,5	1
1	0	−0,5	0	+0,5	1
1	1	+0,5	1	+1,5	1

La domanda giusta: il secondo strato disegna una retta anche lui?

Sì, ma **non nel disegno che stai guardando**. Ed è qui che quasi tutti si perdono, quindi andiamo piano.

Il neurone di uscita riceve *a* e *b*, non *x*₁ e *x*₂. Quindi la sua retta vive in un altro foglio: un foglio i cui assi sono *a* e *b*, non gli amici e i chilometri. Chiamiamolo lo **spazio nascosto**.

Primo strato	Il passaggio	Secondo strato
traccia rette nel foglio degli input (<i>x</i> ₁ , <i>x</i> ₂), quello che vedi tu	ogni punto del foglio degli input viene spedito in un punto del foglio (<i>a</i> , <i>b</i>)	traccia una retta nel foglio (<i>a</i> , <i>b</i>), che è un foglio diverso

Il punto è questo: **una retta dritta nel foglio nascosto, riportata indietro nel foglio di partenza, non è più una retta**. Diventa il bordo di un cuneo, di una striscia, di un poligono. Ecco perché due strati fanno cose che uno strato solo non può fare: non perché sappiano tracciare curve, ma perché tracciano rette in un foglio che il primo strato ha già ridisegnato.

Nella prossima sezione vediamo questo meccanismo all’opera sul problema che ha fatto la storia.

13. Il muro: il problema che una retta non può risolvere

Cambiamo di nuovo la regola di casa:

*Andiamo al cinema se piove **oppure** se ci sono gli amici,
ma se piove **e** ci sono gli amici restiamo a casa a giocare a carte.*

Con $x_1 = \text{piove}$ (0 o 1) e $x_2 = \text{ci sono gli amici}$ (0 o 1):

x_1 piove	x_2 amici	y	
0	0	0	niente da fare, resto a casa
0	1	1	ci sono gli amici: cinema
1	0	1	piove e sono solo: cinema
1	1	0	piove e ci sono gli amici: carte in casa

Questa funzione si chiama **XOR**, cioè “o l’uno o l’altro ma non tutti e due”. Sembra innocua. È stata la tomba del perceptron.

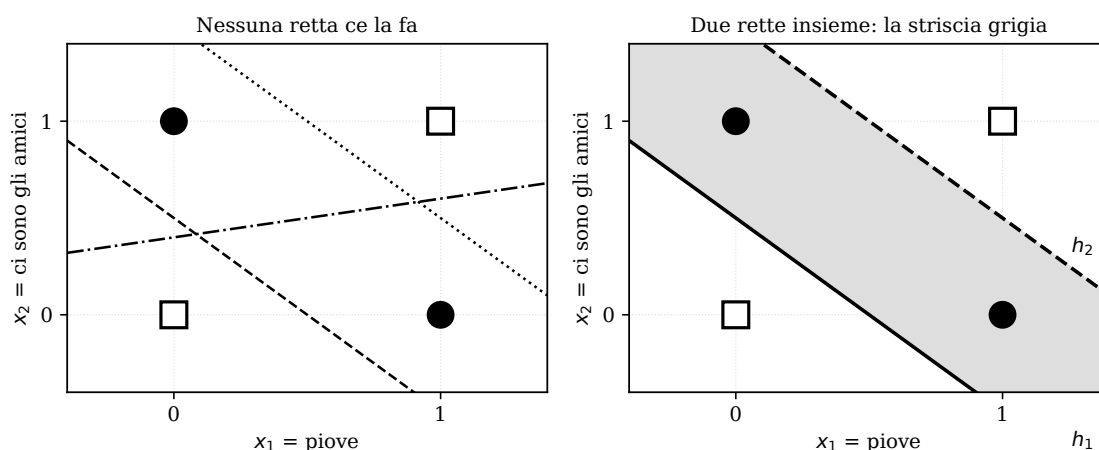


Figura 11: A sinistra: i quattro casi dello XOR. I due cerchi neri stanno su una diagonale, i due quadrati bianchi sull'altra. Le tre rette tratteggiate sono altrettanti tentativi, e nessuno funziona. A destra: con due rette invece si può ritagliare la striscia che contiene esattamente i due cerchi neri.

13.1 La dimostrazione, in quattro righe

Non è che non abbiamo trovato la retta giusta: la retta giusta **non esiste**, ed è facile dimostrarlo. Supponiamo per assurdo che esistano tre numeri w_1 , w_2 e b che funzionano. Allora devono valere tutte e quattro queste condizioni:

$$\begin{array}{lll}
 \text{da } (0,0) \rightarrow 0: & b < 0 & \text{quindi } -b > 0 \\
 \text{da } (1,0) \rightarrow 1: & w_1 + b \geq 0 & \text{quindi } w_1 \geq -b \\
 \text{da } (0,1) \rightarrow 1: & w_2 + b \geq 0 & \text{quindi } w_2 \geq -b \\
 \text{da } (1,1) \rightarrow 0: & w_1 + w_2 + b < 0 &
 \end{array}$$

A MANO Sommiamo la seconda e la terza: $w_1 + w_2 \geq -2b$. Aggiungiamo b da entrambe le parti:

$$w_1 + w_2 + b \geq -2b + b = -b > 0$$

Ma la quarta condizione pretendeva $w_1 + w_2 + b < 0$. Una quantità non può essere insieme maggiore e minore di zero: **assurdo**. Quindi non esiste nessuna scelta di pesi che risolva lo XOR con un solo perceptron. Non è questione di allenarlo di più o di essere più bravi: è matematicamente impossibile.

13.2 La soluzione, e perché funziona

Mettiamo due neuroni nel primo strato, uno che fa la O e uno che fa la E:

$$h_1 = \text{gradino}(x_1 + x_2 - 0,5) \quad (\text{la O}), \quad h_2 = \text{gradino}(x_1 + x_2 - 1,5) \quad (\text{la E})$$

e poi un neurone di uscita:

$$\hat{y} = \text{gradino}(h_1 - h_2 - 0,5)$$

A MANO Compila tu la tabella, riga per riga.

x_1	x_2	z_1	h_1	z_2	h_2	z_{out}	$\hat{y}$	vero y
0	0	-0,5	0	-1,5	0	-0,5	0	0
0	1	+0,5	1	-0,5	0	+0,5	1	1
1	0	+0,5	1	-0,5	0	+0,5	1	1
1	1	+1,5	1	+0,5	1	-0,5	0	0

Quattro righe su quattro. Ma *perché* funziona? Guarda la colonna (h_1, h_2) e confrontala con quella di partenza:

punto di partenza (x_1, x_2)	diventa (h_1, h_2)	risposta voluta
(0, 0)	(0, 0)	0
(0, 1)	(1, 0)	1
(1, 0)	(1, 0)	1
(1, 1)	(1, 1)	0

I due punti che volevamo separare dagli altri sono finiti nello stesso posto. Quattro punti di partenza sono diventati tre punti di arrivo, e in quei tre punti il problema è banale: basta una retta.

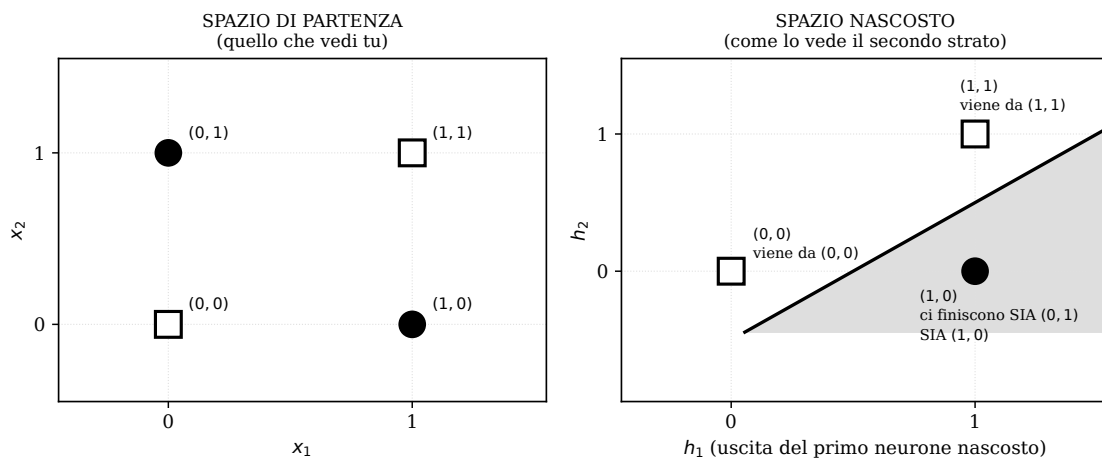


Figura 12: Questa è la figura più importante della dispensa. **A sinistra** lo spazio di partenza, dove i quattro punti non si possono separare con una retta. **A destra** lo spazio nascosto, dopo che il primo strato ha riscritto ogni punto. I punti $(0, 1)$ e $(1, 0)$ sono stati spinti nello stesso posto e adesso la retta $h_2 = h_1 - 0,5$ separa tutto senza fatica.

La frase da ricordare per tutto il resto della vostra vita

Uno strato nascosto non impara a tracciare curve. **Impara a spostare i punti** finché il problema non diventa abbastanza semplice da poter essere risolto con una retta. Il deep learning è questa idea, ripetuta molte volte di seguito.

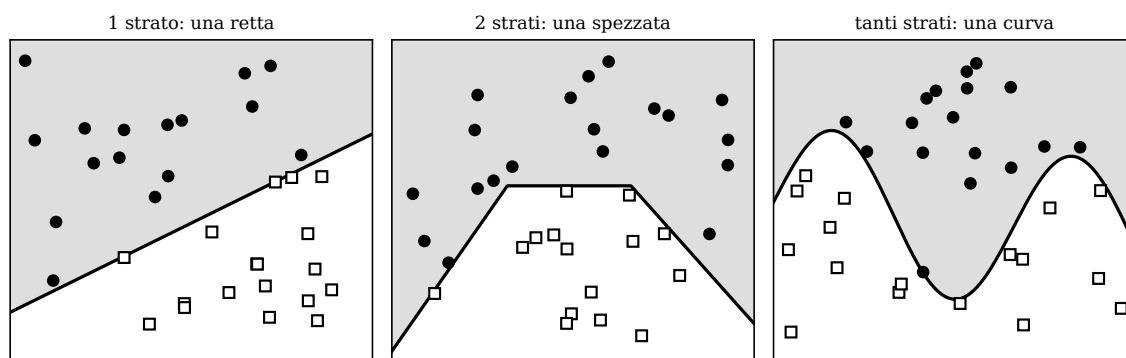


Figura 13: Il risultato visto dal foglio di partenza. Con uno strato la frontiera è una retta; con due diventa una spezzata, perché è fatta di pezzi di rette incollati; con molti strati può assumere qualunque forma.

14. Rosenblatt, Minsky e vent'anni di silenzio

Vale la pena raccontare come andò davvero, perché è una delle storie più istruttive della scienza del Novecento.

Nel 1958 **Frank Rosenblatt** presenta pubblicamente il perceptron, e due anni dopo ne costruisce la versione fisica, il Mark I Perceptron: una macchina grande come un armadio, piena di fili e potenziometri, capace di imparare a riconoscere forme semplici. I giornali si scatenano: il *New York Times* scrive che la Marina americana ha creato l'embrione di un calcolatore che un giorno camminerà, parlerà, vedrà, scriverà e sarà consapevole di sé.

Rosenblatt aveva anche dimostrato un teorema importante, il **teorema di convergenza del perceptron**: se i dati sono separabili da una retta, allora l'algoritmo che hai eseguito a mano *la trova sempre*, in un numero finito di passi. È un bel teorema. Il problema è la premessa, quel "se".

Nel 1969 **Marvin Minsky** e **Seymour Papert**, del MIT, pubblicano il libro *Perceptrons*, in cui dimostrano con rigore matematico proprio quello che hai appena dimostrato tu qualche pagina fa: un perceptron a uno strato non può fare lo XOR, né riconoscere se una figura è connessa, né molte altre cose elementari. La dimostrazione è ineccepibile e il tono, a tratti, sarcastico verso l'entusiasmo esagerato di quegli anni.

Ecco però il punto che spesso si dimentica: Minsky e Papert **sapevano benissimo** che con più strati il problema si risolveva, e lo scrivono nel libro. Quello che aggiungono è che nessuno sapeva come *addestrare* una rete a più strati, e si dichiarano pessimisti sul fatto che qualcuno ci sarebbe riuscito.

Il vero ostacolo, spiegato bene

Il motivo tecnico è tutto nella funzione a gradino, e adesso hai gli strumenti per capirlo fino in fondo.

Con un solo strato, quando la macchina sbaglia sai esattamente chi ha colpa: gli input che erano accesi. La formula $w_i \leftarrow w_i + \eta e x_i$ funziona proprio perché x_i ti dice quanta colpa ha ciascuno.

Con due strati non lo sai più. Se la risposta finale è sbagliata, la colpa è del neurone a ? Del neurone b ? Del peso v_1 ? E di quanto? Il problema si chiama **attribuzione della responsabilità** (in inglese *credit assignment problem*) e con il gradino è insolubile, per un motivo preciso:

la funzione a gradino è piatta dappertutto.

Se cambi un peso nascosto di un centesimo, il neurone nascosto quasi certamente resta sullo stesso scalino, la sua uscita resta identica, e la risposta finale non cambia di niente. L'errore ti dice "sbagliato" ma non ti dice "sbagliato in questa direzione". Non hai nessuna pista da seguire. Provare tutte le combinazioni di pesi è impossibile, perché i pesi sono numeri continui.

L'effetto del libro fu devastante. I finanziamenti alle reti neurali si prosciugarono quasi ovunque e cominciò il periodo che oggi si chiama **il primo inverno dell'intelligenza artificiale**. Rosenblatt stava preparando una soluzione al problema ma morì poco dopo, nel 1971, in un incidente in barca a vela, il giorno del suo quarantatreesimo compleanno. Non partecipò al dibattito che seguì e soprattutto non fece in tempo a mostrare una soluzione. 51 anni dopo all'età di 94 anni avrebbe potuto vedere il frutto del suo perceptron: ChatGpt.

La morale non è che Minsky avesse torto: aveva ragione su tutto quello che ha dimostrato. La morale è che una dimostrazione di impossibilità vale solo sotto le ipotesi che contiene, e che cambiare una sola ipotesi, in questo caso la funzione a gradino, può riaprire una strada che sembrava chiusa per sempre.

Dall'altra parte, la comunità scientifica di oggi attribuisce spesso a Minsky la colpa di aver fatto crollare i finanziamenti alla ricerca sulle reti neurali. Tuttavia, Minsky non riteneva di aver causato un danno, bensì di aver fatto chiarezza scientifica su un campo che, a suo dire, era dominato da proclami sensazionalistici non supportati dai fatti. È importante sottolineare che, nonostante la ferocia del dibattito accademico, Minsky e Rosenblatt erano stati compagni di scuola alla *Bronx High School of Science* e si stimavano a vicenda. La disputa era di natura prettamente scientifica ed epistemologica; di conseguenza, Minsky non ritenne mai di dover porre scuse personali o morali alla famiglia di Rosenblatt.

15. Intermezzo necessario: che cos'è una derivata

Per uscire dall'inverno serve un attrezzo: la derivata. Se non l'hai ancora fatta a scuola, queste due pagine sotto ti basteranno per capire tutto. Se non fossero sufficienti... c'è ChatGpt, Claude, Gemini, DeepSeek o bot simili che ti possono aiutare nello studio.

15.1 L'idea: quanto cambia una cosa se ne muovi un'altra

Hai una quantità E (per noi sarà l'errore) che dipende da un numero w (per noi un peso). La **derivata di E rispetto a w** , che si scrive $\frac{\partial E}{\partial w}$ e si legge “de E su de w ”, risponde a una sola domanda:

se aumento w di un pochino, di quanto cambia E ?

Se viene $+3$, vuol dire che aumentando w di un centesimo, E aumenta di circa tre centesimi. Se viene -3 , E *diminuisce* di tre centesimi. Geometricamente è la **pendenza della retta tangente** al grafico in quel punto.

15.2 Come si calcola: primo metodo, con il righello

Il metodo più onesto è quello della definizione. Prendi un passettino h , guardi quanto cambia E , e dividi:

$$\frac{E(w+h) - E(w)}{h}$$

Questo si chiama **rapporto incrementale**. Poi rimpicciolisci h e vedi dove va a finire il risultato.

A MANO Facciamolo su $E(w) = w^2$ nel punto $w = 1,5$, dove $E(1,5) = 2,25$:

h	$E(1,5+h)$	rapporto incrementale
1	6,2500	$(6,2500 - 2,2500)/1 = 4,00$
0,5	4,0000	$(4,0000 - 2,2500)/0,5 = 3,50$
0,25	3,0625	$(3,0625 - 2,2500)/0,25 = 3,25$
0,1	2,5600	$(2,5600 - 2,2500)/0,1 = 3,10$
0,01	2,2801	$(2,2801 - 2,2500)/0,01 = 3,01$

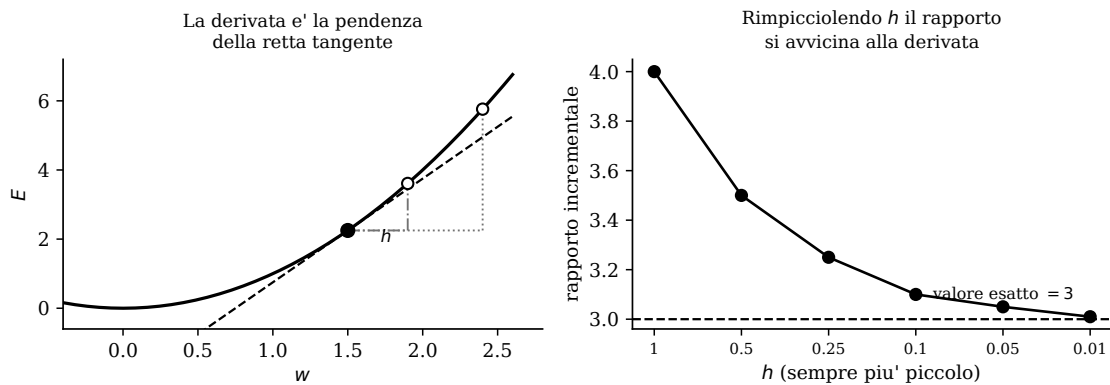


Figura 14: A sinistra: la derivata è la pendenza della retta tangente (tratteggiata). I due triangolini mostrano il rapporto incrementale per due valori di h : più h è piccolo, più la retta che unisce i due punti somiglia alla tangente. A destra: la tabella qui sopra, in forma di grafico.

I numeri si avvicinano a 3. E infatti $3 = 2 \cdot 1,5$: la derivata di w^2 è $2w$.

Questo metodo funziona sempre e sul computer si usa davvero, per controllare che le formule scritte a mano siano giuste. Si chiama *controllo del gradiente* e lo faremo anche noi fra poco.

15.3 Come si calcola: secondo metodo, con le regole

Nella pratica non si fa mai il limite: si imparano cinque regole e si applicano. Sono queste, e ti bastano per tutta la dispensa.

funzione	derivata	esempio
una costante k	0	la derivata di 7 è 0
w	1	
w^n	$n w^{n-1}$	la derivata di w^3 è $3w^2$
$k \cdot f(w)$	$k \cdot f'(w)$	la derivata di $5w^2$ è $10w$
$f(w) + g(w)$	$f'(w) + g'(w)$	la derivata di $w^2 + w$ è $2w + 1$

A MANO Verifica che la derivata di $E = \frac{1}{2}(y - \hat{y})^2$ rispetto a $\hat{y}$, con y fissato, vale $-(y - \hat{y})$. Ecco i passaggi: poni $u = y - \hat{y}$; allora $E = \frac{1}{2}u^2$ e per la regola della potenza $\partial E / \partial u = u$. Ma quando $\hat{y}$ cresce di 1, u cala di 1, quindi $\partial u / \partial \hat{y} = -1$. Moltiplicando: $u \cdot (-1) = -(y - \hat{y})$.

Quel “moltiplicando” non era una furbizia. Era la regola più importante di tutte.

15.4 La regola della catena, cioè il cambio valuta

Se E dipende da u , e u dipende da w , allora

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial u} \cdot \frac{\partial u}{\partial w}$$

Il modo più semplice di ricordarla è pensare ai cambi di valuta. Se un euro vale 1,1 dollari, e un dollaro vale 150 yen, quanti yen vale un euro? Moltiplichi: $1,1 \times 150 = 165$. Le derivate si comportano esattamente così: sono “cambi”, cioè quanto di una cosa ottieni per una unità di un'altra, e si compongono moltiplicando.

Ed ecco perché è la regola più importante: **una rete neurale è una catena di funzioni**. L'errore dipende dall'uscita, che dipende dal punteggio, che dipende dai pesi. Per sapere quanto conta un peso in fondo alla catena, basta moltiplicare i cambi lungo tutto il percorso. Questa e nient'altro è la backpropagation.

16. Il gradiente: l'idea che ha cambiato tutto

Che cosa mancava, esattamente? Mancava un modo di misurare **quanto** si sta sbagliando, non solo *se* si sta sbagliando.

16.1 Primo passo: buttare via la funzione gradino nel cestino (che fa anche rima!)

Sostituiamo il gradino con una curva morbida, la **sigmoide**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Quello che conta è la forma: sale dolcemente da 0 a 1, passando per 0,5 quando $z = 0$. L'uscita non è più una decisione secca ma un **grado di convinzione**: 0,95 vuol dire “quasi sicuro di sì”, 0,5 vuol dire “non ne ho idea”, 0,1 vuol dire “quasi sicuro di no”.

La sua derivata ha una forma comodissima da ricordare:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

z	-3	-2	-1	0	1	2	3
$\sigma(z)$	0,05	0,12	0,27	0,50	0,73	0,88	0,95
$\sigma'(z)$	0,05	0,10	0,20	0,25	0,20	0,10	0,05

A MANO Verifica una casella: $\sigma(1) = 0,73$, quindi $\sigma'(1) = 0,73 \cdot (1 - 0,73) = 0,73 \cdot 0,27 \approx 0,20$.

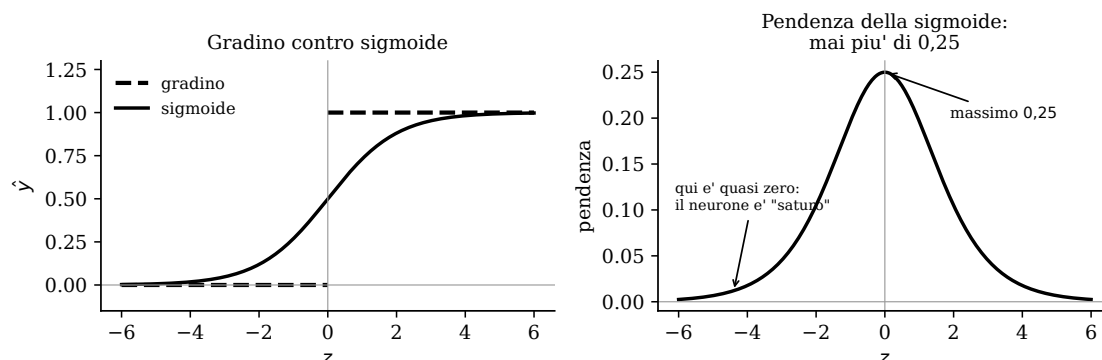


Figura 15: A sinistra la sigmoide confrontata con il gradino. A destra la sua pendenza, che vale al massimo 0,25 e crolla a zero agli estremi. Tieni bene a mente quel numero 0,25: fra tre pagine sarà il protagonista.

16.2 Secondo passo: misurare l'errore con un numero

Definiamo l'errore su un singolo esempio come

$$E = \frac{1}{2} (y - \hat{y})^2$$

Si eleva al quadrato perché così l'errore è sempre positivo (non importa da che parte sbagli) e perché sbagliare di molto pesa molto più che sbagliare di poco. Il mezzo davanti serve solo a far venire i conti più belli quando si deriva.

Adesso E è un numero che cambia con continuità: se sposto un peso di pochissimo, E cambia di pochissimo. **Ed è proprio questo che ci mancava.**

16.3 Terzo passo: seguire la pendenza

Immagina di essere su una collina nella nebbia e di voler scendere a valle. Non vedi niente, ma puoi sentire con i piedi da che parte il terreno scende. Fai un passo in quella direzione, ti fermi, senti di nuovo, fai un altro passo. Prima o poi arrivi in fondo. La collina è l'errore E , la posizione sono i pesi, la pendenza sotto i piedi è il **gradiente**:

$$w_i^{\text{nuovo}} = w_i^{\text{vecchio}} - \eta \cdot \frac{\partial E}{\partial w_i}$$

C'è il segno meno perché l'errore vogliamo farlo *scendere*, quindi ci muoviamo nel verso opposto alla salita.

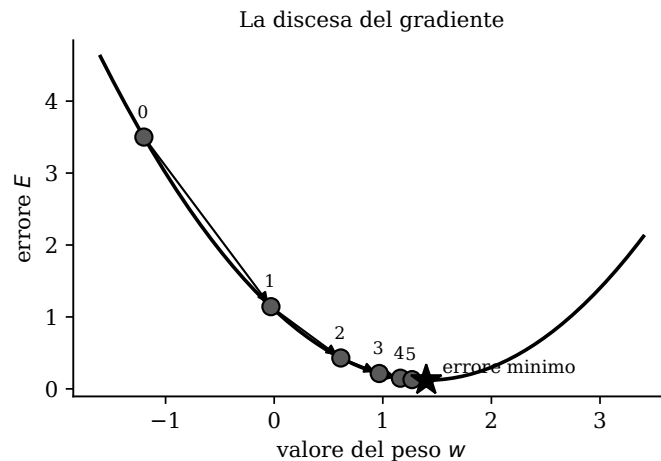


Figura 16: La discesa del gradiente. I passi sono grandi dove la pendenza è forte e diventano piccolissimi vicino al minimo: è il freno automatico incorporato nel metodo.

16.4 Un neurone solo, un passo, tutto a mano

Prendiamo il sabato $x = (1, 2)$ con risposta vera $y = 1$, pesi $w = (0,4; -0,2)$, bias $b = 0$ e $\eta = 1$.

A MANO Passo 1, la somma pesata.

$$z = 0,4 \cdot 1 + (-0,2) \cdot 2 + 0 = 0,4 - 0,4 = 0$$

Passo 2, la sigmoide. $\sigma(0) = 0,5$ e $\sigma'(0) = 0,5 \cdot 0,5 = 0,25$. La macchina risponde $\hat{y} = 0,5$: totalmente indecisa.

Passo 3, l'errore. $E = \frac{1}{2}(1 - 0,5)^2 = \frac{1}{2} \cdot 0,25 = 0,125$

Passo 4, il gradiente con la regola della catena. La catena è $w_1 \rightarrow z \rightarrow \hat{y} \rightarrow E$, quindi si moltiplicano tre cambi:

$$\frac{\partial E}{\partial w_1} = \underbrace{\frac{\partial E}{\partial \hat{y}}}_{-(y-\hat{y})} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{\sigma'(z)} \cdot \underbrace{\frac{\partial z}{\partial w_1}}_{x_1} = -(1 - 0,5) \cdot 0,25 \cdot 1 = -0,125$$

Conviene battezzare il pezzo che si ripete sempre:

$$\delta = -(y - \hat{y}) \cdot \sigma'(z) = -0,125$$

e allora tutti gli altri gradienti sono immediati:

$$\frac{\partial E}{\partial w_2} = \delta \cdot x_2 = -0,125 \cdot 2 = -0,25, \quad \frac{\partial E}{\partial b} = \delta \cdot 1 = -0,125$$

Passo 5, l'aggiornamento. Ricorda il segno meno: sottrarre un numero negativo vuol dire sommare.

$w_1 = 0,4 + 0,125 = \mathbf{0,525}, \quad w_2 = -0,2 + 0,25 = \mathbf{0,05}, \quad b = 0 + 0,125 = \mathbf{0,125}$

Il controllo del gradiente: fidati ma verifica

Come faccio a sapere che $-0,125$ è giusto e non ho sbagliato un segno? Uso il metodo del righello: calcolo E per $w_1 = 0,4$ e per $w_1 = 0,4 + h$, e divido.

h	$E(0,4 + h)$	rapporto
0,4	0,0805	$-0,1112$
0,1	0,1128	$-0,1218$
0,01	0,1238	$-0,1247$

I numeri corrono verso $-0,125$: la formula è giusta. Questo controllo è esattamente quello che fanno i ricercatori quando scrivono una rete nuova, e smaschera quasi tutti gli errori di segno.

16.5 Che cosa cambia rispetto a Rosenblatt

	regola del perceptron (1958)	discesa del gradiente
uscita	solo 0 oppure 1	un numero fra 0 e 1
errore	+1, -1 oppure 0	un numero qualunque, anche 0,043
quando corregge	solo se ha sbagliato del tutto	sempre, anche quando ha ragione per poco
quanti strati	uno solo	quanti se ne vuole
garanzia	trova la retta se esiste	scende sempre, ma può fermarsi in una conca

17. La backpropagation, passo per passo

Adesso il pezzo grosso: come si addestra una rete a due strati. Lo faremo su una rete piccolissima ma vera, con numeri scelti perché i conti vengano esatti.

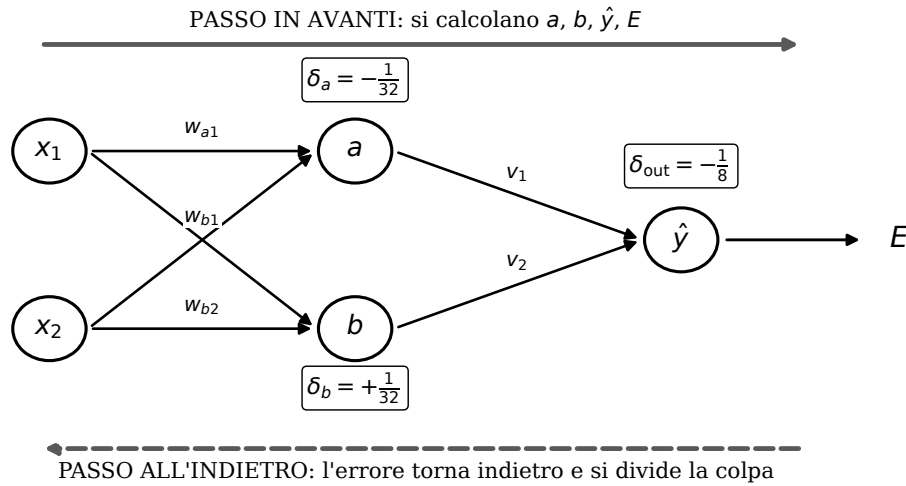


Figura 17: La rete dell'esempio e i due passaggi. Prima si va avanti per calcolare la risposta e l'errore, poi si torna indietro per distribuire la colpa.

La rete ha due input, due neuroni nascosti (a e b) e un neurone di uscita. Tutti usano la sigmoide. I numeri di partenza:

	peso su x_1	peso su x_2	bias
neurone a	$w_{a1} = 0,5$	$w_{a2} = -0,25$	$b_a = 0$
neurone b	$w_{b1} = -0,5$	$w_{b2} = 0,25$	$b_b = 0$
	peso su a	peso su b	bias
uscita	$v_1 = 1$	$v_2 = -1$	$c = 0$

Esempio: $x = (1, 2)$ con risposta vera $y = 1$. Tasso di apprendimento $\eta = 2$ (grande, ma così i numeri restano belli).

17.1 Fase 1: il passo in avanti

A MANO Si calcola tutto da sinistra a destra.

$$\begin{aligned}
 z_a &= 0,5 \cdot 1 + (-0,25) \cdot 2 + 0 = 0,5 - 0,5 = 0 & \Rightarrow a &= \sigma(0) = \frac{1}{2} \\
 z_b &= -0,5 \cdot 1 + 0,25 \cdot 2 + 0 = -0,5 + 0,5 = 0 & \Rightarrow b &= \sigma(0) = \frac{1}{2} \\
 z_{\text{out}} &= 1 \cdot \frac{1}{2} + (-1) \cdot \frac{1}{2} + 0 = 0 & \Rightarrow \hat{y} &= \sigma(0) = \frac{1}{2}
 \end{aligned}$$

L'errore è $E = \frac{1}{2} \left(1 - \frac{1}{2}\right)^2 = \frac{1}{8} = 0,125$.

Le tre pendenze che ci serviranno valgono tutte $\sigma'(0) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$.

17.2 Fase 2: il passo all'indietro

Adesso si torna da destra a sinistra. La domanda a cui rispondiamo per ogni neurone è sempre la stessa: *di quanto cambierebbe l'errore se il punteggio z di questo neurone fosse un pochino diverso?* Chiamiamo δ questa risposta.

Il neurone di uscita.

$$\delta_{\text{out}} = -(y - \hat{y}) \cdot \sigma'(z_{\text{out}}) = -\left(1 - \frac{1}{2}\right) \cdot \frac{1}{4} = -\frac{1}{2} \cdot \frac{1}{4} = -\frac{1}{8} = -0,125$$

Da qui i gradienti dei pesi dell'ultimo strato escono subito, perché come sempre si moltiplica il δ per quello che entrava nel collegamento:

$$\frac{\partial E}{\partial v_1} = \delta_{\text{out}} \cdot a = -\frac{1}{8} \cdot \frac{1}{2} = -\frac{1}{16}, \quad \frac{\partial E}{\partial v_2} = \delta_{\text{out}} \cdot b = -\frac{1}{16}, \quad \frac{\partial E}{\partial c} = -\frac{1}{8}$$

I neuroni nascosti: qui avviene la magia. Quanta colpa ha il neurone a ? Ragiona come faresti tu: a ha influito sull'errore *solo passando attraverso* il neurone di uscita, e il canale che usava era il peso v_1 . Quindi si prende il δ di chi sta a valle, lo si moltiplica per il peso del collegamento e poi per la propria pendenza. È di nuovo la regola della catena, i cambi valuta che si moltiplicano:

$$\delta_a = \underbrace{\delta_{\text{out}}}_{\text{colpa a valle}} \cdot \underbrace{v_1}_{\text{quanto passa dal collegamento}} \cdot \underbrace{\sigma'(z_a)}_{\text{la mia pendenza}} = -\frac{1}{8} \cdot 1 \cdot \frac{1}{4} = -\frac{1}{32} = -0,03125$$

$$\delta_b = -\frac{1}{8} \cdot (-1) \cdot \frac{1}{4} = +\frac{1}{32} = +0,03125$$

Nota il segno di δ_b : è opposto, perché il collegamento verso l'uscita ha peso -1 . Il neurone b deve muoversi nella direzione contraria rispetto ad a per ottenere lo stesso effetto. La rete lo capisce da sola, senza che nessuno glielo dica.

E adesso i gradienti dei pesi del primo strato, sempre con la stessa formula “ δ per quello che entra”:

$$\frac{\partial E}{\partial w_{a1}} = \delta_a \cdot x_1 = -\frac{1}{32} \cdot 1 = -\frac{1}{32}, \quad \frac{\partial E}{\partial w_{a2}} = \delta_a \cdot x_2 = -\frac{1}{32} \cdot 2 = -\frac{1}{16}, \quad \frac{\partial E}{\partial b_a} = -\frac{1}{32}$$

17.3 Fase 3: si aggiorna tutto

A MANO Con $\eta = 2$, e ricordando che si *sottrae* il gradiente:

$$\begin{aligned} v_1 &= 1 - 2 \cdot \left(-\frac{1}{16}\right) = 1 + \frac{1}{8} = \mathbf{1,125} & v_2 &= -1 + \frac{1}{8} = \mathbf{-0,875} & c &= 0 + \frac{1}{4} = \mathbf{0,25} \\ w_{a1} &= 0,5 + \frac{1}{16} = \mathbf{0,5625} & w_{a2} &= -0,25 + \frac{1}{8} = \mathbf{-0,125} & b_a &= 0 + \frac{1}{16} = \mathbf{0,0625} \\ w_{b1} &= -0,5 - \frac{1}{16} = \mathbf{-0,5625} & w_{b2} &= 0,25 - \frac{1}{8} = \mathbf{0,125} & b_b &= 0 - \frac{1}{16} = \mathbf{-0,0625} \end{aligned}$$

17.4 Fase 4: ha funzionato?

Rifacciamo il passo in avanti con i pesi nuovi:

$$\begin{aligned} z_a &= 0,5625 - 0,25 + 0,0625 = 0,375 & \Rightarrow a &= \sigma(0,375) \approx 0,593 \\ z_b &= -0,5625 + 0,25 - 0,0625 = -0,375 & \Rightarrow b &= \sigma(-0,375) \approx 0,407 \\ z_{\text{out}} &= 1,125 \cdot 0,593 - 0,875 \cdot 0,407 + 0,25 \approx 0,560 & \Rightarrow \hat{y} &\approx 0,637 \end{aligned}$$

$$E = \frac{1}{2}(1 - 0,637)^2 \approx \mathbf{0,066}$$

Era 0,125, adesso è 0,066: quasi dimezzato in un solo passo, e la risposta è salita da 0,50 a 0,64, cioè verso l'1 che volevamo.

Il riassunto in tre righe

- Avanti:** calcola tutte le uscite fino all'errore.
- Indietro:** parti dall'ultimo δ e propagalo verso sinistra moltiplicando ogni volta per il peso del collegamento e per la pendenza del neurone.
- Aggiorna:** il gradiente di un peso è sempre il δ del neurone dove il collegamento arriva, per il valore che il collegamento portava.

Questo è tutto. Una rete con cento miliardi di pesi usa esattamente queste tre righe.

18. Il secondo ostacolo: quando il gradiente svanisce

La backpropagation viene diffusa nel 1986 da **David Rumelhart**, **Geoffrey Hinton** e **Ronald Williams** con un articolo su *Nature* (l'idea era già comparsa prima, per esempio nella tesi di Paul Werbos del 1974, ma è quell'articolo che convince il mondo). Il problema di Minsky è risolto. Le reti a due o tre strati cominciano a funzionare.

E allora perché il deep learning è arrivato solo venticinque anni dopo?

Perché appena si prova ad aggiungere strati, salta fuori un problema nuovo, e la sua spiegazione sta già nei conti che hai appena fatto a mano.

18.1 Il conto che spiega tutto

Riguarda questo passaggio della backpropagation:

$$\delta_a = \delta_{\text{out}} \cdot v_1 \cdot \sigma'(z_a)$$

Abbiamo ottenuto $-\frac{1}{8} \rightarrow -\frac{1}{32}$, cioè il segnale si è **ridotto a un quarto** attraversando un solo strato. Non è un caso: è colpa di σ' , che come hai visto nella tabella **non supera mai 0,25**.

A MANO Adesso immagina di ripetere quel passaggio dieci volte, cioè di avere dieci strati:

$$0,25^{10} = 0,00000095 \approx \frac{1}{1\,000\,000}$$

strati	quanto resta del gradiente	
1	0,25	un quarto
2	0,0625	un sedicesimo
5	0,00098	un millesimo
10	0,00000095	un milionesimo
20	0,00000000000091	praticamente zero

I primi strati non ricevono più nessun segnale. I loro pesi restano quasi identici a quelli casuali di partenza, per sempre. La rete è profonda solo sulla carta: in pratica imparano solo gli ultimi strati. Questo si chiama **problema del gradiente che svanisce** (in inglese *vanishing gradient*).

E può andare anche peggio nell'altro verso. Se i pesi sono grandi, diciamo che ogni strato moltiplica per 3 invece che per 0,25, allora dopo venti strati il fattore è $3^{20} \approx 3,5$ miliardi: il gradiente **esplode**, i pesi schizzano a valori assurdi e l'addestramento va in pezzi. È il **gradiente che esplode** (*exploding gradient*).

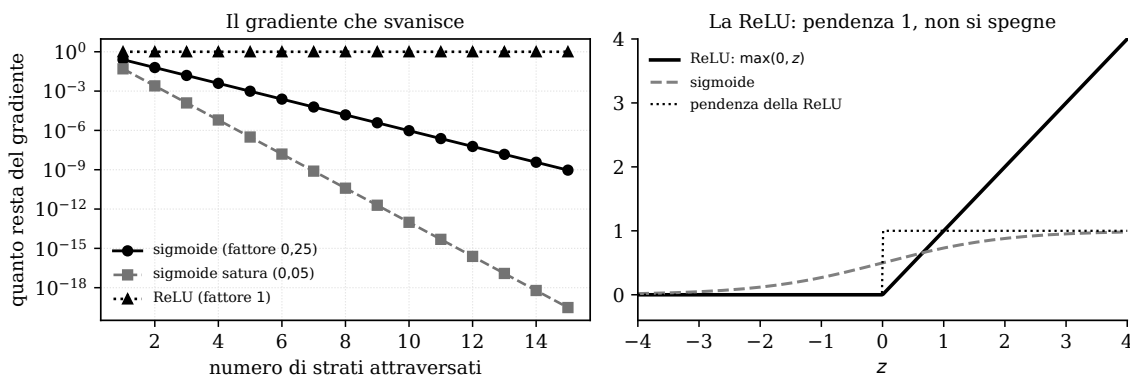


Figura 18: A sinistra, quanto resta del gradiente dopo n strati (nota la scala verticale: ogni tacca vale dieci volte la precedente). A destra, la funzione ReLU e la sua pendenza, che vale 1 e quindi non riduce niente.

18.2 Chi lo ha capito e chi lo ha risolto

Il problema viene identificato con precisione nel 1991 da **Sepp Hochreiter** nella sua tesi di laurea, e viene analizzato a fondo nel 1994 in un articolo di **Yoshua Bengio**, Patrice Simard e Paolo Frasconi che dimostra che con la discesa del gradiente imparare dipendenze lunghe è intrinsecamente difficile. Bengio è un nome da tenere a mente perché lo ritroveremo altre due volte in queste pagine.

Le soluzioni sono arrivate una alla volta, e sono quasi tutte semplici:

rimedio	chi e quando	che cosa fa
ReLU $\max(0, z)$	Glorot, Bordes e Bengio, 2011	la pendenza vale 1 invece di 0,25: il segnale non si riduce più
inizializzazione furba	Glorot e Bengio 2010, poi He 2015	si scelgono i pesi iniziali con la scala giusta perché il segnale non esploda né si spenga
normalizzazione	Ioffe e Szegedy, 2015	si rimettono in scala le uscite di ogni strato durante l'addestramento
connessioni residue	He e altri, 2015	si aggiungono scorciatoie che saltano gli strati, così il gradiente ha una strada dritta per tornare indietro

La ReLU merita un secondo di attenzione perché è la più bella: è la funzione più stupida che si possa immaginare, $\max(0, z)$, cioè “se il numero è negativo scrivi zero, altrimenti lascialo stare”. La sua pendenza vale 1 per $z > 0$, e 1 moltiplicato mille volte fa ancora 1. Il problema più grave del deep learning è stato risolto in buona parte da una funzione che si spiega in mezza riga.

19. Dal perceptron al deep learning

Adesso possiamo raccontare la storia intera.

Deep learning (apprendimento profondo) vuol dire usare reti con molti strati. Ma non è vero, come si sente dire, che sia bastato aggiungere strati: aggiungere strati era esattamente la cosa che non funzionava. Sono servite tre categorie di scoperte.

Prima categoria: i trucchi per far imparare le reti profonde

Sono quelli della tabella qui sopra, più altri. Senza ReLU, senza inizializzazione corretta e senza connessioni residue, una rete di cinquanta strati semplicemente non impara. Questo è il contributo tecnico che ha permesso tutto il resto.

Seconda categoria: architetture pensate per il tipo di dato

Una rete in cui ogni neurone è collegato a tutti quelli dello strato precedente si chiama *completamente connessa*, ed è quella che hai calcolato a mano. Su un'immagine da un milione di pixel avrebbe miliardi di pesi solo nel primo strato: ingestibile.

Yann LeCun, alla fine degli anni Ottanta, propone le **reti convoluzionali**: invece di un peso diverso per ogni pixel, lo stesso gruppetto di pesi (un “filtro”) scorre su tutta l'immagine. Con qualche centinaio di pesi si riconosce un bordo ovunque esso sia. Nel 1989 questa idea legge già i codici di avviamento postale sulle buste per il servizio postale americano.

Yoshua Bengio, nel 2003, fa la stessa mossa sul linguaggio con *A Neural Probabilistic Language Model*. L'idea è che ogni parola venga rappresentata non da un simbolo, ma da una lista di numeri (oggi la chiamiamo *embedding*), in modo che parole simili finiscano vicine. È l'invenzione dei modelli linguistici neurali, cioè i nonni diretti degli LLM di oggi.

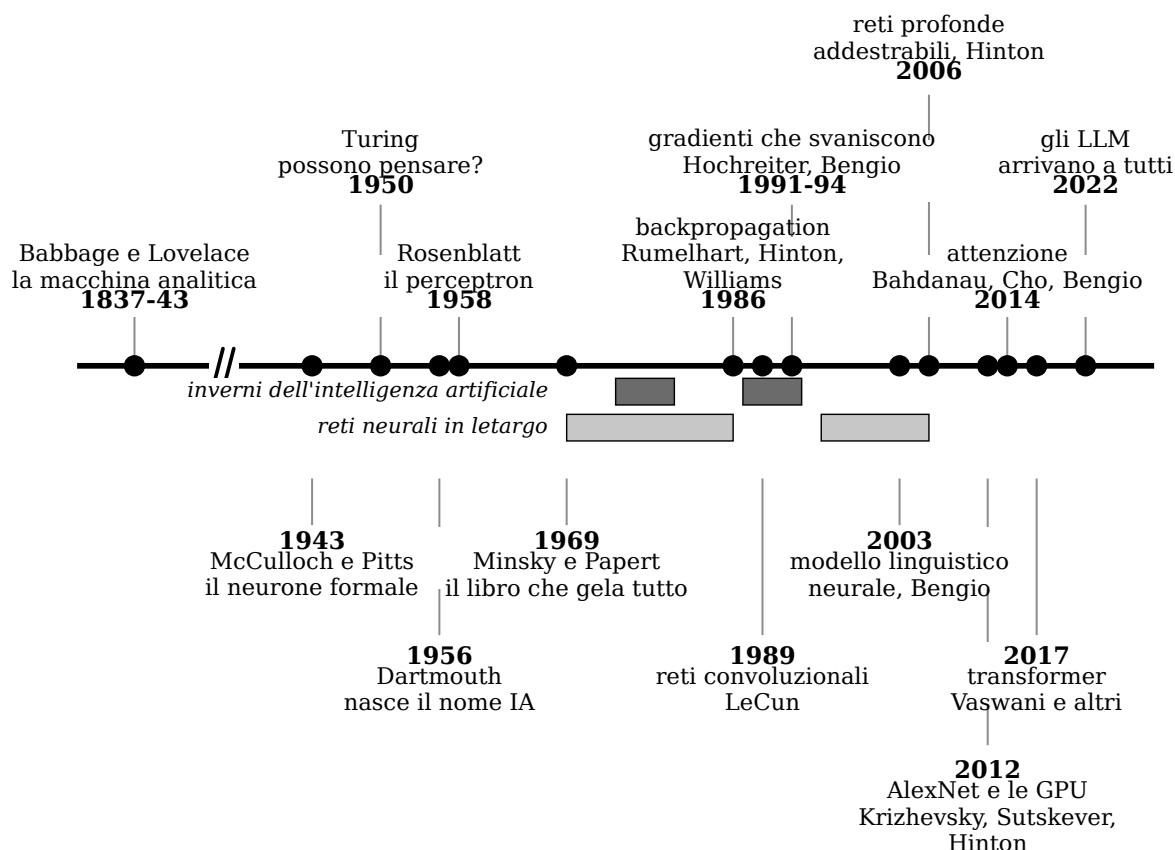


Figura 19: Quasi due secoli di AI. Il segno // sulla linea è un'interruzione dell'asse: fra il 1843 e il 1943 la scala è compressa di circa otto volte, perché in quei cent'anni, per questa storia, non succede nulla. Nessun evento cade nel tratto tagliato, ed è proprio questa la condizione che rende lecito tagliarlo: le distanze fra i punti disegnati restano confrontabili fra loro. Senza il taglio, all'era moderna resterebbe meno della metà della larghezza invece dell'ottantaquattro per cento. **Sotto l'asse ci sono due fasce e non una**, ed è il punto della figura: gli *inverni dell'intelligenza artificiale* (grigio scuro) e i *letarghi delle reti neurali* (grigio chiaro) sono crisi diverse, con cause diverse, che si sovrappongono soltanto in mezzo. Il primo inverno dell'IA comincia cinque anni dopo il letargo delle reti e finisce sei anni prima: confonderli, come fa quasi tutta la divulgazione, fa perdere il senso di entrambi. In ogni etichetta l'anno in grassetto è la riga più vicina alla linea del tempo e il nome di chi c'era sta subito oltre; il filo sottile collega l'etichetta al suo punto, e si interrompe dove passa dietro a un'altra etichetta.

Terza categoria: i dati e le macchine

Nel 2012 **Alex Krizhevsky**, **Ilya Sutskever** e **Geoffrey Hinton** presentano AlexNet, una rete convoluzionale addestrata su ImageNet (un milione e duecentomila fotografie etichettate) usando due schede grafiche da videogioco. Stravince la gara annuale di riconoscimento di immagini con un distacco che nessuno si aspettava. È il momento in cui il mondo cambia idea.

Nel 2018 Bengio, Hinton e LeCun ricevono insieme il premio Turing, che è l'equivalente del Nobel per l'informatica. Nel 2024 Hinton e Hopfield hanno vinto il Nobel per la Fisica **per le scoperte fondamentali sull'apprendimento automatico con reti neurali artificiali**, mentre lo stesso anno gli sviluppatori di **AlphaFold** hanno ottenuto il Nobel per la Chimica per aver risolto una sfida cinquantennale della biologia, **prevedendo con l'IA la struttura tridimensionale di quasi tutte le proteine conosciute** per accelerare la ricerca medica.

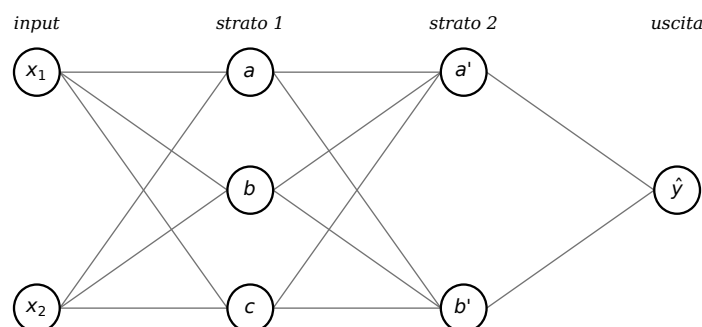


Figura 20: Una rete con due strati nascosti. Ogni cerchio è un perceptron identico a quello che hai calcolato a mano nelle prime pagine: moltiplica, somma, decide. Le reti di oggi sono questa figura con decine o centinaia di colonne, e con migliaia di cerchi per colonna.

20. Perché hanno vinto le reti, e non altri metodi

Negli anni Novanta e nei primi Duemila le reti neurali non erano affatto il metodo migliore. Il campione erano le **macchine a vettori di supporto** (Support Vector Machine, SVM), inventate da Vladimir Vapnik e colleghi. Erano più eleganti, più solide teoricamente e in pratica vincevano quasi sempre. Poi hanno perso. Vale la pena capire perché, perché la risposta spiega tutta l'epoca in cui viviamo.

Che cosa fa una SVM

Una SVM cerca la retta di separazione *migliore possibile*, cioè quella con il margine più largo dai punti più vicini. E quando i dati non sono separabili da una retta, usa un trucco raffinatissimo detto *kernel*: calcola le somiglianze tra tutte le coppie di esempi e fa finta di lavorare in uno spazio enormemente più grande, dove una retta basta.

L'idea è splendida. Il guaio è nel "tutte le coppie".

Primo motivo: il costo cresce con il quadrato dei dati

Se hai n esempi, le coppie sono circa $n^2/2$. Con mille esempi sono mezzo milione di conti: niente. Con dieci milioni di esempi sono cinquanta *mila miliardi*: impossibile, e non basterebbe neanche la memoria per scrivere la tabella.

Una rete neurale invece si addestra a **piccoli gruppi** (mini-batch): prende 32 esempi, fa un passo di gradiente, li butta via, ne prende altri 32. Il costo di un passo non dipende da quanti dati hai in tutto, quindi il costo totale cresce *proporzionalmente* al numero di esempi, non al suo quadrato. Puoi addestrare su dati che non entrerebbero mai nella memoria del computer.

Secondo motivo: le reti sembrano essere state fatte apposta per le schede grafiche usate nei videogiochi: le GPU

Questo è il motivo decisivo. Guarda di nuovo che cosa fa uno strato di rete: ogni neurone calcola $w_1x_1 + w_2x_2 + \dots$, e tutti i neuroni lo fanno *contemporaneamente*, sugli stessi input, senza aspettarsi a vicenda. Mettere in fila i pesi di tutti i neuroni significa che uno strato intero è una **moltiplicazione fra matrici**, e nient'altro.

Una scheda grafica è nata per calcolare dove finiscono milioni di triangoli sullo schermo di un videogioco, il che è ancora una moltiplicazione fra matrici. Ha migliaia di piccoli processori che fanno tutti la stessa operazione su dati diversi. Il risultato è che una GPU addestra una rete cento volte più in fretta di un processore normale, e il salto è arrivato gratis, pagato dall'industria dei videogiochi.

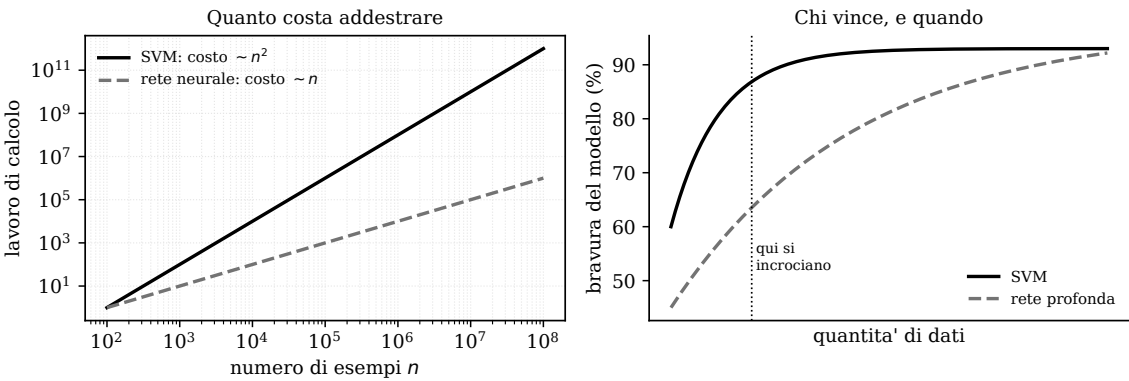


Figura 21: A sinistra: come cresce il lavoro di calcolo (entrambi gli assi hanno tacche che si moltiplicano per dieci). A destra: chi è più bravo. Con pochi dati la SVM vince, ed è ancora oggi una scelta ottima quando gli esempi sono poche migliaia. Con molti dati la rete continua a migliorare e sorpassa.

L’ottimizzazione di una SVM invece è un processo largamente sequenziale, fatto di scelte che dipendono dalle precedenti: si parallelizza molto peggio.

Terzo motivo: chi sceglie le caratteristiche

Con una SVM devi decidere tu il kernel, cioè come misurare la somiglianza fra due esempi. Su dati semplici è un vantaggio, perché ci metti dentro quello che sai. Su una fotografia è una condanna: nessuno sa scrivere a mano la formula che dice quando due immagini contengono lo stesso gatto.

Una rete profonda **impara la rappresentazione da sola**, ed è esattamente la cosa che hai visto succedere nella figura dello spazio nascosto dello XOR. Con tanti dati, imparare a mano perde sempre contro imparare dai dati.

	SVM	rete profonda
costo con n esempi	circa n^2	circa n
parallelizzabile	poco	moltissimo, sono matrici
rappresentazione	la scegli tu (il kernel)	la impara la rete
con pochi dati	spesso migliore	rischia di imparare a memoria
con moltissimi dati	non ci arriva proprio	continua a migliorare

21. Dai modelli linguistici all’attenzione

Restava un ultimo problema, ed era il linguaggio.

Per tradurre una frase servono le reti *ricorrenti*: leggono una parola alla volta e si portano dietro un riassunto di quello che hanno letto. Queste cosiddette RNN (Recurrent Neural Network) hanno due problemi molto grossi. Il primo lo conosci già: leggendo parola per parola, il gradiente deve tornare indietro attraverso tutta la frase e svanisce, quindi la rete si dimentica l’inizio (è proprio il problema studiato da Hochreiter e da Bengio). Il secondo è che leggere in ordine è **sequenziale**, quindi butta via il vantaggio delle schede grafiche: non puoi elaborare la parola numero cinquanta prima di aver finito la quarantanove.

2014: l’attenzione

Dzmitry Bahdanau, Kyunghyun Cho e Yoshua Bengio propongono una soluzione bellissima. Invece di comprimere tutta la frase in un unico riassunto, permettono alla rete, a ogni parola che produce, di **guardare indietro a tutte le parole di partenza e decidere a quali dare peso**. Quel meccanismo si chiama **attenzione**.

Ed ecco il punto che ci riguarda: l'attenzione è ancora una somma pesata.

$$\text{risultato} = \alpha_1 \cdot (\text{parola}_1) + \alpha_2 \cdot (\text{parola}_2) + \dots$$

La differenza rispetto al perceptron è una sola, ma è enorme: nel perceptron i pesi w_i sono fissi e imparati una volta per tutte, mentre nell'attenzione i pesi α_i **vengono ricalcolati per ogni frase**, in base a quanto ogni parola somiglia a quella che sto cercando. Sono pesi che cambiano al volo.

2017: tutto quello che ti serve è l'attenzione

Nel 2017 otto ricercatori guidati da **Ashish Vaswani** pubblicano un articolo con un titolo che è diventato leggendario: *Attention Is All You Need*, cioè "l'attenzione è tutto quello che serve".

L'idea è radicale: se l'attenzione funziona così bene, buttiamo via la lettura sequenziale e teniamo solo l'attenzione. La rete che ne esce si chiama **transformer** e guarda tutte le parole insieme, in parallelo.

Il vantaggio non è solo di qualità, è soprattutto di velocità: senza sequenza si possono usare a fondo le schede grafiche, quindi si possono addestrare modelli enormi su tutto il testo di internet. Da lì in poi la strada verso i modelli linguistici di oggi (GPT, Claude, Gemini, DeepSeek e gli altri) è stata soprattutto una questione di scala.

22. Che cosa resta del perceptron dentro un LLM

Chiudiamo il cerchio. Quando scrivi una domanda a un modello linguistico, che cosa succede davvero dentro? Molto di quello che hai calcolato a mano.

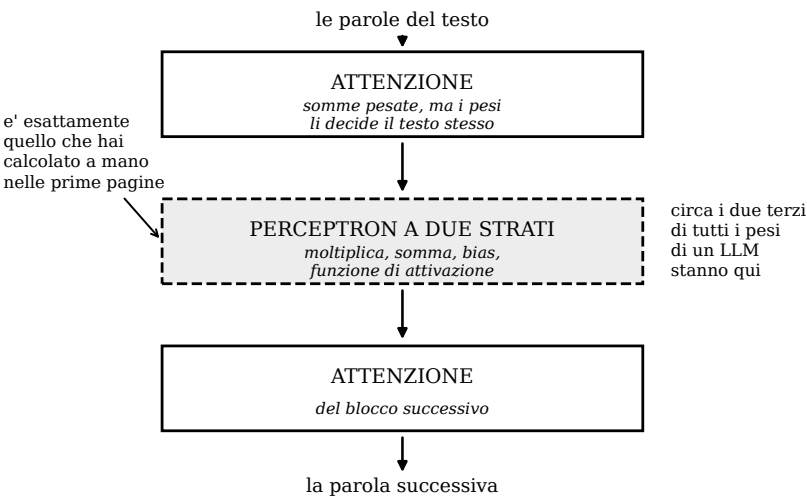


Figura 22: Un blocco di transformer, semplificato. Un modello vero ne impila decine o centinaia. Il riquadro tratteggiato è, letteralmente, un perceptron a due strati.

nel perceptron	nel transformer	che cosa è cambiato
$z = \sum w_i x_i + b$	lo stesso, dentro ogni testa di attenzione e in ogni strato	niente: è ancora moltiplica e somma
pesi w_i fissi	nell'attenzione i pesi si ricalciano per ogni frase	i pesi diventano dinamici
funzione a gradino	ReLU, GELU e parenti	servono per avere una pendenza da seguire
la sigmoide dà un numero fra 0 e 1	la <i>softmax</i> dà una probabilità per ognuna delle parole possibili	è la sigmoide generalizzata a molte scelte
regola di Rosenblatt	discesa del gradiente con backpropagation	si corregge sempre, non solo quando si sbaglia
3 numeri da imparare	centinaia di miliardi	solo la scala

Il pezzo che sorprende di più è quello nel riquadro tratteggiato della figura. In ogni blocco di un transformer, subito dopo l'attenzione, c'è un modulo chiamato *feed-forward*: prende i numeri che escono, li moltiplica per una matrice di pesi, aggiunge un bias, applica una funzione di attivazione, rimoltiplica per un'altra matrice. È esattamente, senza nessuna differenza concettuale, il perceptron a due strati che hai calcolato a mano poche pagine fa. E in un modello linguistico moderno **circa due terzi di tutti i pesi stanno lì dentro**.

Il conto che vale la pena fare

Nella riga R2 dell'epoca 1 hai fatto tre moltiplicazioni e tre somme. Un modello linguistico grande, durante il suo addestramento, esegue la stessa operazione elementare all'incirca 10^{24} volte. Se la facessi tu a mano, un conto al secondo, senza mai dormire, ci metteresti circa trentamila miliardi di miliardi di anni. Non è cambiata l'operazione: è cambiato solo quanto in fretta la sappiamo ripetere.

E che cosa non c'era nel perceptron

Per onestà, tre cose davvero nuove ci sono, e nessuna è ovvia:

1. i **pesi che si calcolano al volo** dell'attenzione, che non hanno nessun corrispettivo in Rosenblatt;
2. l'idea di **imparare la rappresentazione** invece di sceglierla, che è la vera eredità di LeCun e Bengio;
3. il fatto che addestrare su tantissimo testo a indovinare la parola successiva produca, come effetto collaterale, la capacità di fare cose per cui nessuno ha addestrato il modello. Questo non lo aveva previsto nessuno e in buona parte non lo sappiamo ancora spiegare.

23. Come funziona un LLM, pezzo per pezzo

Nella sezione precedente abbiamo detto *che cosa* resta del perceptron dentro un modello linguistico. Adesso lo smontiamo davvero, pezzo per pezzo, e facciamo i conti a mano su una frase vera. Alla fine di questa sezione avrai eseguito, con carta e penna, tutte le operazioni che un modello come quelli che usi ogni giorno esegue miliardi di volte al secondo.

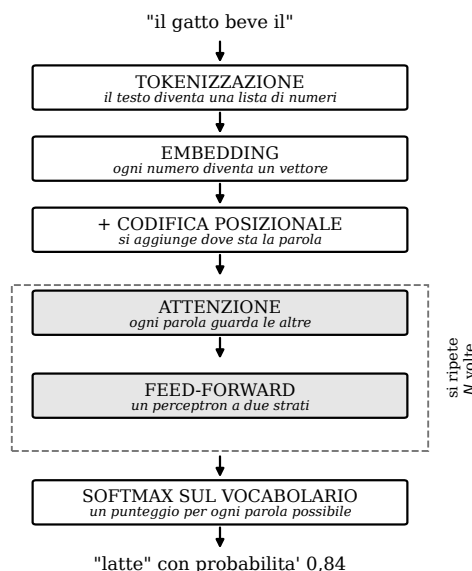


Figura 23: La pila completa. Ogni riquadro è una sezione di questo capitolo. Il riquadro tratteggiato si ripete decine o centinaia di volte in un modello vero.

23.1 Il compito: indovinare la parola dopo

Un modello linguistico fa una cosa sola, e più stupida di quanto sembri:

data una sequenza di parole, dire quale parola viene dopo.

Nient'altro. Quando gli fai una domanda, il modello non “risponde”: continua il testo, una parola alla volta, e ogni parola che produce viene riattaccata in fondo e usata per produrre la successiva. Tutta l'apparente intelligenza esce da questo gioco ripetuto milioni di volte.

Il nostro esempio sarà questo:

“il gatto beve il ...” → ?

23.2 Primo pezzo: dal testo ai token

Il computer non capisce le lettere, capisce i numeri. Serve quindi un dizionario che associ a ogni pezzo di testo un numero. Ma quali pezzi?

- **Una lettera alla volta?** Il vocabolario sarebbe piccolissimo (una sessantina di simboli), ma le sequenze diventerebbero lunghissime e il modello dovrebbe reimparare da zero che “g-a-t-t-o” vuol dire qualcosa.
- **Una parola intera?** Le sequenze sarebbero corte, ma il vocabolario enorme e soprattutto *aperto*: appena arriva una parola mai vista, per esempio un cognome o “gattinamente”, il modello non saprebbe che farsene.

La soluzione è una via di mezzo: si spezza il testo in **token**, pezzi di parola scelti in modo che quelli frequenti restino interi e quelli rari si spezzino. Il metodo più usato si chiama **BPE** (*Byte Pair Encoding*) e il suo funzionamento è talmente semplice che lo puoi eseguire a mano.

L' algoritmo BPE in tre righe

1. Parti spezzando tutto in singole lettere.

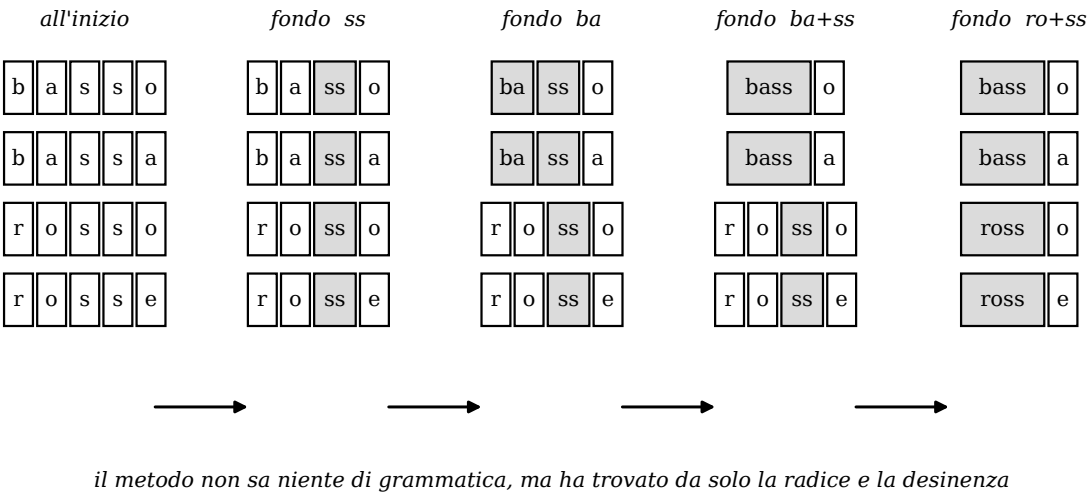
2. Conta tutte le coppie di simboli vicini e trova la coppia più frequente.

3. Fondi quella coppia in un simbolo nuovo. Torna al punto 2 e ripeti finché il vocabolario non ha la dimensione voluta.

A MANO Facciamolo su un mini-testo di quattro parole: *basso, bassa, rosso, rosse*. Conta tu le coppie del primo passo.

coppia	quante volte	
s + s	4	compare in tutte e quattro le parole
b + a	2	
a + s	2	
r + o	2	
o + s	2	
s + o	2	

Vince *ss*, che diventa un simbolo unico. Si ricomincia, e le fusioni successive sono *ba*, poi *ba+ss = bass*, poi *ro* e infine *ro+ss = ross*.



→

→

→

→

il metodo non sa niente di grammatica, ma ha trovato da solo la radice e la desinenza

Figura 24: Le prime fusioni del BPE. Le caselle grigie sono i token nati dalle fusioni.

Guarda il risultato: *bass|o, bass|a, ross|o, ross|e*. L'algoritmo ha separato la radice dalla desinenza. Nessuno gli ha mai spiegato che cosa sia una radice o una desinenza: ha solo contato le coppie frequenti, e la struttura della lingua è emersa da sola. È lo stesso fenomeno che hai visto nel perceptron quando ha scoperto da solo che la distanza scoraggia.

Nel nostro esempio le parole sono comuni, quindi ogni parola è un token unico:

token	il	gatto	beve	latte	cane	mangia	pane
numero	1	2	3	4	5	6	7

La nostra frase diventa dunque la lista di numeri [1, 2, 3, 1]. Nota che i due “il” hanno lo stesso numero: per adesso il modello non ha modo di distinguerli. Ci torneremo.

23.3 Token speciali, padding e maschere

Un modello non elabora una frase alla volta ma un **gruppo** di frasi insieme (si chiama *batch*), per sfruttare fino in fondo la scheda grafica. Ma le frasi hanno lunghezze diverse, e una scheda grafica vuole tabelle rettangolari.

La soluzione è il **padding**: si sceglie la lunghezza della frase più lunga del gruppo e si riempiono le altre con un token finto, chiamato appunto `<PAD>`, che di solito ha il numero 0.

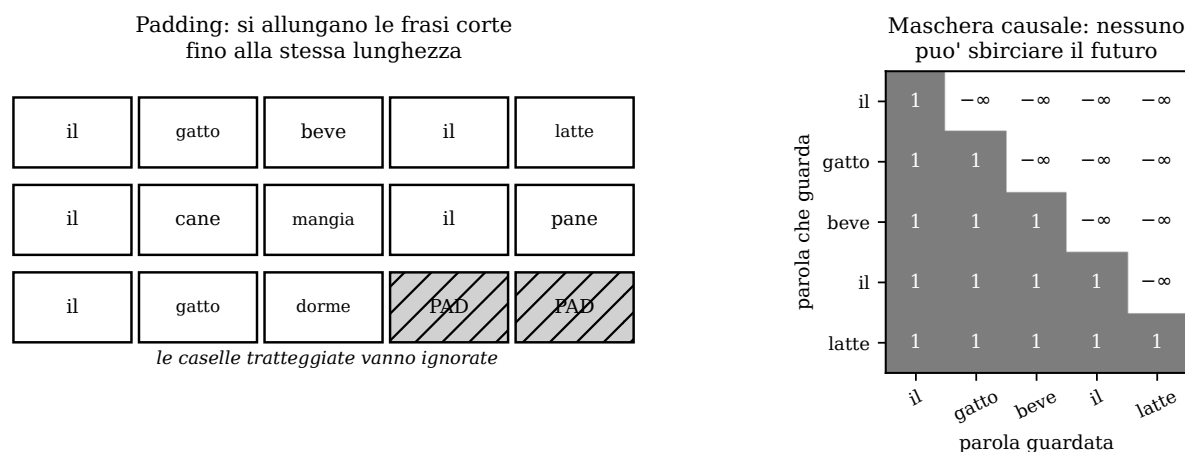


Figura 25: A sinistra il padding, a destra la maschera causale.

Ma poi bisogna dire al modello di ignorare quei token finti, altrimenti imparerebbe sciocchezze. Si usa una **maschera**: una lista di 1 e 0 lunga quanto la frase, dove 0 vuol dire “qui non c’è niente”. Nel punto in cui il modello calcola i punteggi (fra due pagine) i punteggi delle posizioni mascherate vengono messi a $-\infty$.

A MANO Perché proprio $-\infty$? Perché più avanti quei punteggi finiscono dentro un esponenziale, e

$$e^{-\infty} = 0$$

quindi quella posizione riceve peso esattamente zero. Non è un trucco sporco: è il modo pulito di dire “questa casella non esiste”.

Esiste anche una seconda maschera, molto più importante, detta **maschera causale**: mentre il modello impara a indovinare la parola successiva, non deve poter sbirciare le parole che vengono dopo, altrimenti il compito sarebbe banale. Si mette dunque a $-\infty$ tutto ciò che sta a destra, ottenendo la forma a triangolo della figura. È esattamente il motivo per cui un LLM scrive da sinistra a destra e non può tornare indietro a correggersi.

Ci sono poi altri token speciali: `<BOS>` e `<EOS>` per segnare l’inizio e la fine di un testo, e `<UNK>` per l’ignoto.

23.4 Secondo pezzo: l’embedding

Adesso abbiamo dei numeri, per esempio gatto = 2. Possiamo darli in pasto alla rete così come sono?

A MANO No, e vale la pena capire perché. Se “gatto” vale 2 e “beve” vale 3, la rete calcolerebbe cose come $w \cdot 2$ e $w \cdot 3$, e dedurrebbe che “beve” è una specie di “gatto” più grande del cinquanta per cento. Ma quei numeri erano solo etichette messe a caso: non c’è nessun ordine fra le parole del dizionario.

La soluzione è dare a ogni parola non un numero ma un **vettore**, cioè una lista di numeri. Questa lista si chiama **embedding** (in italiano si dice talvolta *immersione*), e ogni sua casella misura quanto quella parola possiede una certa proprietà.

Nel nostro esempio useremo vettori di sole quattro caselle:

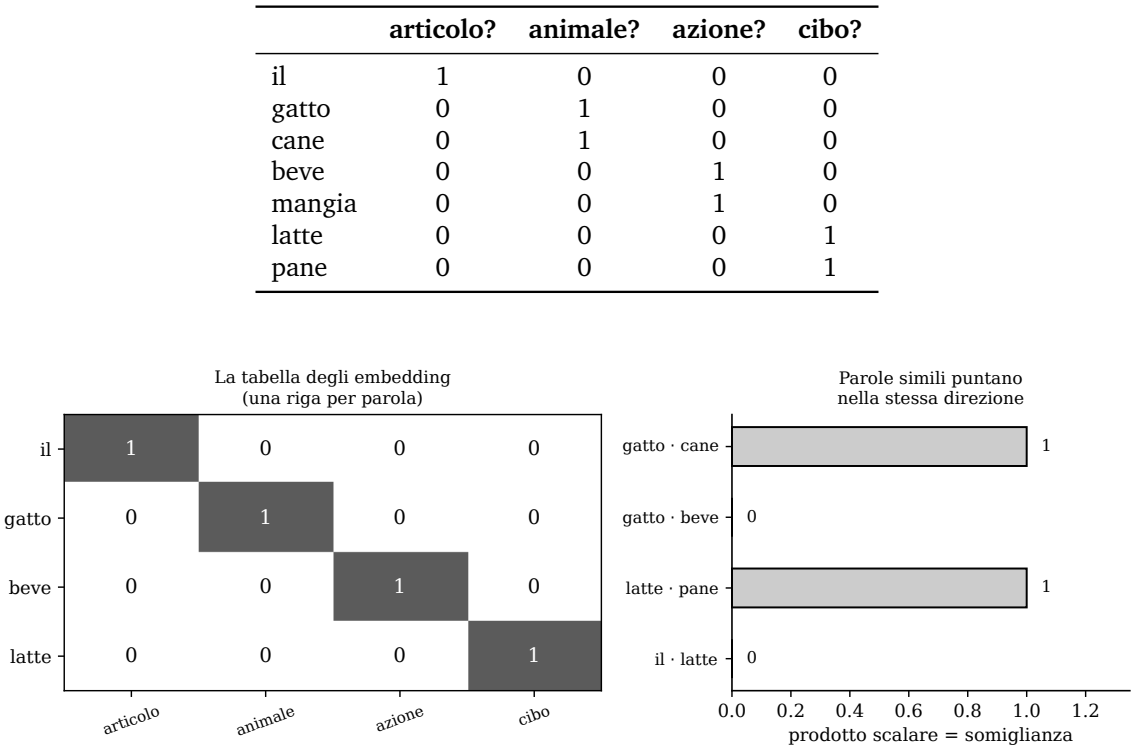


Figura 26: A sinistra la tabella degli embedding: è una semplice tabella di consultazione, una riga per parola. A destra la somiglianza fra due parole, misurata con il prodotto scalare.

Come si misura se due parole sono simili

Con il **prodotto scalare**, che è la stessa somma pesata del perceptron: si moltiplicano le caselle corrispondenti e si somma.

A MANO gatto · cane = $0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 = 1$: molto simili.

A MANO gatto · beve = $0 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 = 0$: niente in comune.

E adesso il punto importante, quello che spiega perché i modelli veri sono così grandi. Nella nostra tabella **gatto e cane hanno lo stesso identico vettore**. Con quattro caselle non c'è modo di distinguerli: il modello sa che sono entrambi animali e basta. Lo stesso vale per latte e pane, e per beve e mangia.

Per distinguerli servono più caselle: una per “domestico”, una per “fa le fusa”, una per “liquido”, e così via. Un modello vero usa vettori di **migliaia** di caselle, e nessuna di esse ha un nome scelto da un umano: la rete decide da sola quali proprietà le convengono, esattamente come il perceptron decideva da solo il segno del peso della distanza.

Le tabelle di consultazione sono pesi anche loro

La tabella degli embedding non è scritta a mano da nessuno: è fatta di numeri che partono **casuali** e vengono imparati con la discesa del gradiente, come tutti gli altri pesi. All'inizio del suo addestramento un modello non sa che gatto e cane si assomigliano; lo scopre perché compaiono nelle stesse frasi, e la discesa del gradiente avvicina i loro vettori. Questa è l'idea che Yoshua Bengio ha introdotto nel 2003 e che ha reso possibile tutto il resto.

23.5 Terzo pezzo: l'ordine delle parole, con seno e coseno

C'è un problema che sembra piccolo e invece è grosso. Il transformer guarda tutte le parole *insieme*, in parallelo: è proprio questo che lo rende veloce. Ma allora, per come l'abbiamo costruito finora, non ha modo di sapere in che ordine stanno. Per lui

“il gatto beve il latte” e “il latte beve il gatto”

sono la stessa identica cosa: gli stessi vettori, mescolati. E le due frasi non vogliono dire affatto la stessa cosa.

Serve dunque un modo di scrivere “io sono la parola numero 3” dentro il vettore stesso. La soluzione adottata dai transformer è di sommare a ogni embedding un secondo vettore che dipende *solo dalla posizione*: la **codifica posizionale**.

Perché non basta scrivere il numero della posizione

L’idea ingenua sarebbe aggiungere una casella con dentro 0, 1, 2, 3, ... Non funziona, per tre motivi:

1. su un testo lungo quel numero diventa enorme e schiaccia tutte le altre caselle, che valgono circa 1;
2. se addestri su frasi corte e poi ne arriva una lunga, il modello incontra numeri mai visti e non sa che farsene;
3. quello che serve al modello non è tanto la posizione assoluta quanto la **distanza** fra due parole, e da una sottrazione di numeri grandi quella informazione si ricava male.

L’idea: due orologi che girano a velocità diverse

Servono numeri che stiano sempre fra -1 e $+1$ e che non si ripetano troppo presto. Il seno e il coseno fanno esattamente questo.

Immagina un orologio con due lancette: una veloce, che fa un quarto di giro a ogni parola, e una lenta, che ne fa un ottavo. Per ogni posizione segni dove puntano tutte e due. Con due lancette hai quattro numeri, cioè seno e coseno di ciascuna.

A MANO Compila tu la tabella. La lancetta veloce gira di 90° per parola, quella lenta di 45° . Ti servono solo i valori che sai già: $\sin 0^\circ = 0$, $\sin 45^\circ = 0,71$, $\sin 90^\circ = 1$, $\cos 0^\circ = 1$, $\cos 45^\circ = 0,71$, $\cos 90^\circ = 0$.

posizione	lancetta veloce (90°)		lancetta lenta (45°)	
	sin	cos	sin	cos
0	0	+1	0	+1
1	+1	0	+0,71	+0,71
2	0	-1	+1	0
3	-1	0	+0,71	-0,71
4	0	+1	0	-1

Guarda la riga della posizione 4: la lancetta veloce è tornata esattamente dov’era alla posizione 0, ma quella lenta no. È questo che permette di distinguere le due posizioni. Con dieci lancette di velocità sempre più basse si distinguono decine di migliaia di posizioni, e i numeri restano sempre fra -1 e $+1$.

La proprietà che rende tutto questo intelligente

A MANO Calcola il prodotto scalare fra la posizione 1 e la posizione 0:

$$[1, 0, 0,71, 0,71] \cdot [0, 1, 0, 1] = 0 + 0 + 0 + 0,71 = 0,71$$

Adesso fra la posizione 3 e la posizione 2:

$$[-1, 0, 0,71, -0,71] \cdot [0, -1, 1, 0] = 0 + 0 + 0,71 + 0 = 0,71$$

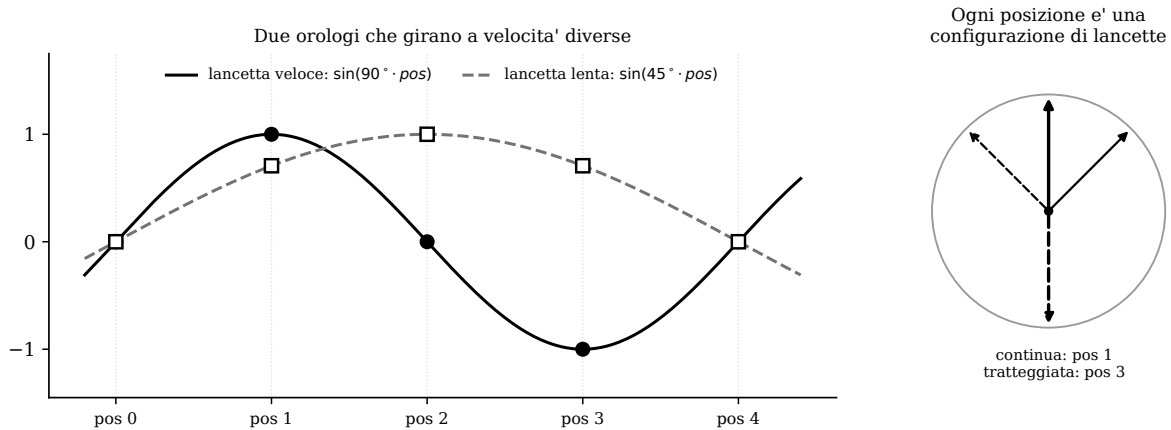


Figura 27: Le due lancette. A sinistra come onde, a destra come orologio. La lancetta veloce da sola tornerebbe al punto di partenza ogni quattro parole; è la combinazione con quella lenta a rendere ogni posizione riconoscibile.

Stesso risultato. E non è un caso: entrambe le coppie sono distanti una posizione. Si può dimostrare in generale che il prodotto scalare fra due codifiche posizionali dipende *soltanto dalla differenza* fra le due posizioni, e non da dove si trovino nella frase:

$$PE(p) \cdot PE(q) = \cos(90^\circ(p - q)) + \cos(45^\circ(p - q))$$

(È la formula di sottrazione del coseno, $\cos \alpha \cos \beta + \sin \alpha \sin \beta = \cos(\alpha - \beta)$, applicata a ogni coppia di caselle.)

Questo è il motivo profondo della scelta di seno e coseno: il modello, che lavora sempre con prodotti scalari, riceve gratis l'informazione “queste due parole sono distanti tre posizioni”, che è proprio quello che serve alla grammatica.

La formula vera

Nei transformer la velocità della lancetta numero i non è scelta a occhio ma data da

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

dove d è quante caselle ha il vettore. Con $d = 4$ i due divisori sono $10000^0 = 1$ e $10000^{1/2} = 100$: la prima lancetta gira cento volte più in fretta della seconda. Le velocità scendono in progressione geometrica, come le cifre di un numero scritto in una base strana. La nostra versione con 90° e 45° è la stessa identica idea con angoli più comodi da fare a mente.

23.6 Quarto pezzo: l'attenzione, con i conti

Siamo al cuore del transformer. Il problema da risolvere è questo: nella frase “il gatto beve il ...”, l'ultimo “il” da solo non dice niente. Per indovinare che cosa viene dopo bisogna *andare a guardare* il verbo “beve”.

L'attenzione fa esattamente questo, e lo fa con un meccanismo che assomiglia a una ricerca in un archivio. Ogni parola produce tre vettori diversi, ottenuti moltiplicando il proprio vettore per tre tabelle di pesi imparate (W_Q, W_K, W_V):

nome	simbolo	a che serve
query (interrogazione)	q	che cosa sto cercando
key (chiave)	k	che cosa offro io a chi cerca
value (valore)	v	che cosa consegno se vengo scelto

L'immagine giusta è quella di una biblioteca. Tu arrivi con una richiesta (*query*); ogni libro ha un'etichetta sul dorso (*key*); confronti la richiesta con tutte le etichette, e prendi il contenuto (*value*) dei libri che corrispondono meglio. La differenza rispetto a una biblioteca vera è che non prendi un libro solo: prendi un po' di tutti, in proporzione a quanto corrispondono.

Il conto, passo per passo

Nel nostro esempio le tre tabelle sono minuscole. Le chiavi e le interrogazioni avranno due sole caselle, che possiamo leggere come “sto cercando un'azione” e “sto cercando un soggetto”. I valori saranno gli embedding stessi.

Le tabelle imparate risultano essere queste:

	W_Q (chi cerca)			W_K (chi si offre)	
	cerco azione	cerco sogg.		sono azione	sono sogg.
articolo	1	0	articolo	0	0
animale	0	1	animale	0	1
azione	0	0	azione	2	0
cibo	0	0	cibo	0	0

A MANO **Passo 1: l'interrogazione.** Chi cerca è l'ultima parola, l'“il” in posizione 3, il cui embedding è $[1, 0, 0, 0]$. Moltiplicando per W_Q si prende la riga “articolo”:

$$q = [1, 0] \quad \text{cioè: sto cercando un'azione}$$

Ha senso: dopo un articolo ci si aspetta un nome, e per sapere *quale* nome bisogna guardare il verbo.

A MANO **Passo 2: le chiavi e i punteggi.** Ogni parola della frase offre la sua chiave, e il punteggio è il prodotto scalare con l'interrogazione.

pos	parola	chiave k	il conto $q \cdot k$	punteggio
0	il	$[0, 0]$	$1 \cdot 0 + 0 \cdot 0$	0
1	gatto	$[0, 1]$	$1 \cdot 0 + 0 \cdot 1$	0
2	beve	$[2, 0]$	$1 \cdot 2 + 0 \cdot 0$	2
3	il	$[0, 0]$	$1 \cdot 0 + 0 \cdot 0$	0

A MANO **Passo 3: la softmax.** I punteggi vanno trasformati in pesi che sommano a 1. Si usa la **softmax**, che è la sorella maggiore della sigmoide: si eleva e a ogni punteggio e si divide per la somma.

$$\text{peso}_i = \frac{e^{\text{punteggio}_i}}{\sum_j e^{\text{punteggio}_j}}$$

Ti serve solo $e^0 = 1$ e $e^2 = 7,39$:

$$\text{somma} = 1 + 1 + 7,39 + 1 = 10,39$$

$$\text{pesi} = \left[\frac{1}{10,39}, \frac{1}{10,39}, \frac{7,39}{10,39}, \frac{1}{10,39} \right] = [0,096; 0,096; \mathbf{0,711}; 0,096]$$

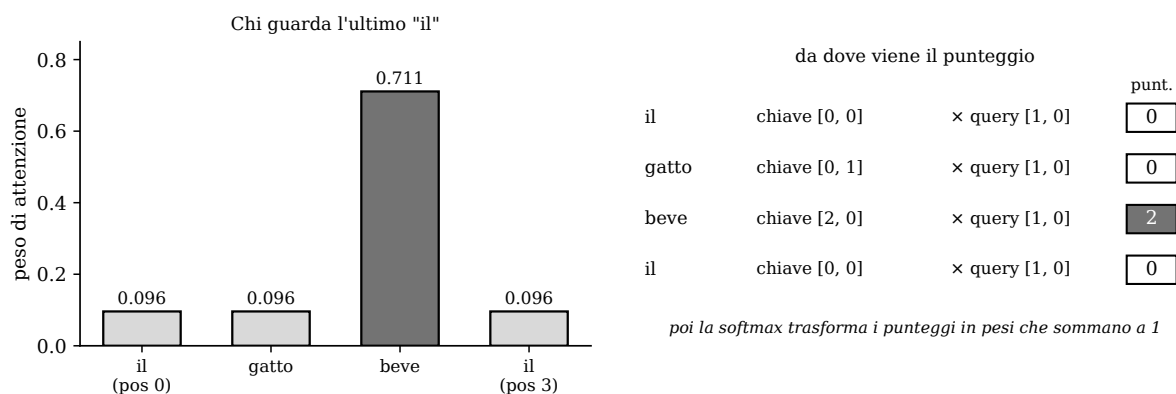


Figura 28: I pesi di attenzione dell'ultimo "il". Il 71% della sua attenzione va sul verbo "beve". Nessuno gliel'ha detto: è il risultato delle tabelle W_Q e W_K imparate.

Perché proprio l'esponenziale

Perché fa tre cose insieme che servono tutte. Primo, rende ogni numero positivo, e un peso negativo non avrebbe senso. Secondo, esagera le differenze: un punteggio di 2 contro uno di 0 diventa un rapporto di sette a uno, quindi il modello sceglie con decisione invece di spalmare. Terzo, è derivabile ovunque, quindi ci si può fare la discesa del gradiente sopra. È esattamente lo stesso ragionamento che ha portato a sostituire il gradino con la sigmoide.

Nei modelli veri i punteggi vengono anche divisi per $\sqrt{d_k}$ (la radice del numero di caselle della chiave) prima della softmax: senza quella divisione, con vettori lunghi i punteggi diventerebbero enormi, l'esponenziale metterebbe tutto il peso su una sola parola e il gradiente si spegnerebbe. È di nuovo il problema del gradiente che svanisce, che ti perseguita anche qui.

A MANO **Passo 4: la media pesata.** Adesso si prendono i valori (per noi, gli embedding) e si mescolano con quei pesi.

$$\begin{aligned}
 \text{uscita} &= 0,096 [1, 0, 0, 0] + 0,096 [0, 1, 0, 0] + 0,711 [0, 0, 1, 0] + 0,096 [1, 0, 0, 0] \\
 &= [0,096; 0; 0; 0] + [0; 0,096; 0; 0] + [0; 0; 0,711; 0] + [0,096; 0; 0; 0] \\
 \text{uscita} &= [0,192; 0,096; \mathbf{0,711}; 0]
 \end{aligned}$$

Leggila: la casella dell'"azione" vale 0,711, tutte le altre sono piccole. L'ultimo "il", che da solo non sapeva niente, adesso porta con sé l'informazione "vicino a me c'è un verbo". Questo è tutto ciò che fa l'attenzione: permette a ogni parola di andarsi a prendere l'informazione che le manca, ovunque si trovi nella frase.

Perché si chiama a più teste

Un modello vero non fa questo conto una volta sola, ma dieci o cento volte in parallelo con tabelle W_Q , W_K , W_V diverse. Ogni copia si chiama **testa** e si specializza: una segue i verbi, una gli accordi di genere e numero, una i riferimenti dei pronomi. I risultati vengono poi affiancati. Le teste non si aspettano a vicenda, quindi girano tutte insieme sulla scheda grafica: è un'altra faccia del vantaggio di parallelizzazione della sezione 20.

23.7 Quinto pezzo: il feed-forward, cioè il tuo perceptron

Il vettore che esce dall'attenzione passa adesso in un modulo che conosci già benissimo, perché lo hai calcolato a mano venti pagine fa: un **perceptron a due strati**.

A MANO Nel nostro esempio ha un solo neurone nascosto, che legge la casella dell'azione con soglia 0,5:

$$h = \text{ReLU}(0,711 - 0,5) = \text{ReLU}(0,211) = \mathbf{0,211}$$

e uno strato di uscita che con peso 10 riversa quel valore sulla casella del cibo:

$$\text{uscita} = [0; 0; 0; 10 \cdot 0,211] = [0; 0; 0; \mathbf{2,11}]$$

La regola che questo perceptron ha imparato, tradotta in italiano, è: *se qui vicino c'è un verbo, allora aspettati un cibo*. Il modulo feed-forward è la memoria del modello: è lì che stanno immagazzinate le regolarità del mondo, mentre l'attenzione si limita a spostare informazione. E, come detto nella sezione precedente, in un LLM vero circa due terzi di tutti i pesi stanno proprio qui dentro.

23.8 Sesto pezzo: dalla pila alla previsione

Adesso bisogna passare da un vettore di quattro caselle a una probabilità per ciascuna delle sette parole del vocabolario. Il modo più elegante, ed è quello che usano i modelli veri, si chiama **legatura degli embedding**: si riusa la tabella degli embedding, calcolando il prodotto scalare fra l'uscita e il vettore di ogni parola. Il ragionamento è: *la parola giusta è quella che punta nella stessa direzione di quello che sto cercando*.

A MANO L'uscita è $[0; 0; 0; 2,11]$, cioè punta tutta nella direzione “cibo”.

parola	suo vettore	punteggio	$e^{\text{punteggio}}$	probabilità
il	[1, 0, 0, 0]	0	1	0,046
gatto	[0, 1, 0, 0]	0	1	0,046
beve	[0, 0, 1, 0]	0	1	0,046
latte	[0, 0, 0, 1]	2,11	8,27	0,384
cane	[0, 1, 0, 0]	0	1	0,046
mangia	[0, 0, 1, 0]	0	1	0,046
pane	[0, 0, 0, 1]	2,11	8,27	0,384
		somma	21,53	1,000

Il modello risponde: *latte* con probabilità 38%, *pane* con probabilità 38%, tutto il resto sotto il 5%.

È un risultato notevole e insieme un fallimento, ed è utile capire entrambe le cose. Notevole perché il modello ha capito benissimo che dopo “il gatto beve il...” viene un cibo: le cinque parole sbagliate hanno tutte probabilità bassa. Fallimento perché non sa scegliere fra latte e pane, e non per stupidità: nella nostra tabella *latte* e *pane* hanno lo stesso identico vettore. Il modello non può distinguere ciò che non ha modo di rappresentare.

23.9 Che cosa fa l'addestramento

Come si misura quanto la risposta è sbagliata? Con la **perdita di entropia incrociata**, che per fortuna nel nostro caso si riduce a una formula semplicissima:

$$L = -\ln P(\text{parola giusta})$$

A MANO Nel nostro caso $L = -\ln(0,384) = \mathbf{0,96}$.

Leggila così: se il modello fosse certissimo e avesse ragione, $P = 1$ e $L = -\ln 1 = 0$, perdita nulla. Se avesse dato alla parola giusta una probabilità quasi nulla, L schizzerebbe verso l'infinito. La perdita punisce soprattutto l'essere sicuri e sbagliati, che è precisamente il difetto che si vuole togliere a un modello.

Addestriamo su tre frasi

Il modello impara su moltissime frasi. Prendiamone tre:

	frase	contesto	parola giusta
F1	il gatto beve il latte	il gatto beve il	latte
F2	il cane mangia il pane	il cane mangia il	pane
F3	il gatto mangia il pane	il gatto mangia il	pane

A MANO Con la tabella degli embedding a quattro caselle, quanto vale la perdita sulla frase F2? Il ragionamento è identico: “mangia” ha lo stesso vettore di “beve”, quindi il conto è lo stesso, e anche la risposta è la stessa: 38% a latte e 38% a pane. Perdita 0,96 di nuovo.

Ecco il punto: **la perdita non può scendere sotto 0,96**, per quanto si addestri, perché il modello non ha abbastanza caselle per distinguere i casi. Si dice che il modello è *in saturazione di capacità*: non è che impara male, è che non ha dove mettere quello che dovrebbe imparare.

Che cosa fa la discesa del gradiente

La discesa del gradiente agisce su tutti i pesi contemporaneamente, embedding compresi. E siccome la perdita resta alta finché latte e pane sono indistinguibili, il gradiente spinge proprio dove serve: **allarga la tabella degli embedding in una direzione nuova**. Immaginiamo che la quinta casella diventi “liquido”:

	articolo	animale	azione	cibo	liquido
beve	0	0	1	0	+1
mangia	0	0	1	0	-1
latte	0	0	0	1	+1
pane	0	0	0	1	-1

A MANO Rifai il conto dell’attenzione: i pesi sono gli stessi di prima, quindi l’uscita adesso è

[0,192; 0,096; 0,711; 0; **0,711**]

perché il 71% del peso sta su “beve”, che nella quinta casella ha +1. Il feed-forward, oltre a quello che faceva prima, lascia passare la quinta casella moltiplicata per 3, dando uscita [0; 0; 0; 2,11; 2,13]. I punteggi diventano:

parola	il conto	punteggio	probabilità
latte	$2,11 \cdot 1 + 2,13 \cdot (+1)$	+4,25	0,848
pane	$2,11 \cdot 1 + 2,13 \cdot (-1)$	-0,02	0,012
beve		+2,13	0,103
gli altri		0 o meno	sotto 0,013

Perdita: $-\ln(0,848) = \mathbf{0,17}$, contro 0,96 di prima.

Questa è, in miniatura, la ragione per cui i modelli sono diventati enormi. Non perché qualcuno pensi che più grande sia più intelligente, ma perché ogni distinzione che il modello deve saper fare ha bisogno di un posto dove stare. Il mondo di distinzioni da fare ne contiene moltissime.

Una nota di onestà su questo esempio

Nel conto qui sopra il modello dà anche il 10% a “beve”, che dopo “beve” non ha alcun senso. Non è un errore di calcolo: è che il nostro feed-forward ha un solo neurone e non ha modo di dire “non ripetere il verbo”. Un modello vero ha migliaia di neuroni per strato proprio per poter imparare anche i divieti, non solo le associazioni. Ho lasciato il difetto invece di nascondere perché mostra bene che cosa si compra con la dimensione.

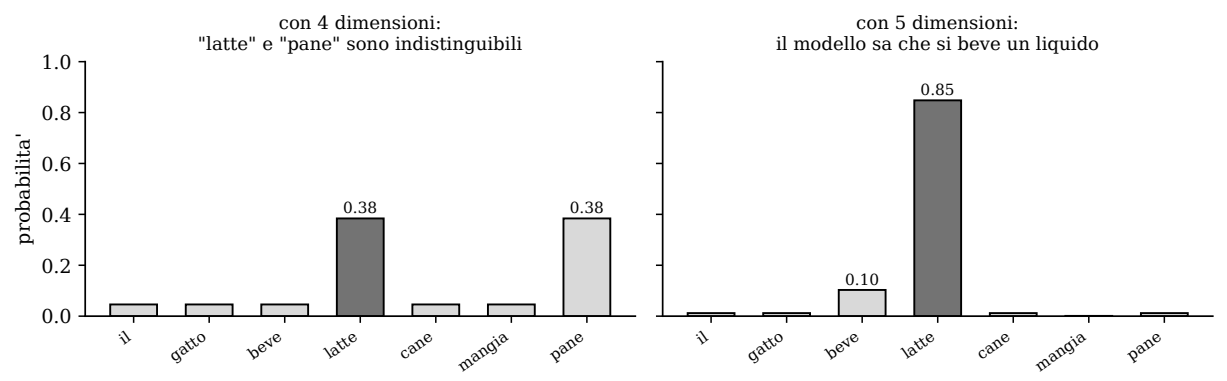


Figura 29: La stessa domanda, prima e dopo. L’unica differenza è una casella in più nella tabella degli embedding.

23.10 Dalla nostra rete a un modello vero

Ecco tutto quello che cambia, e non è concettualmente nulla.

il nostro esempio		un LLM vero
token nel vocabolario	7	da 30 000 a 200 000
caselle per vettore	4	da 2 000 a 16 000
teste di attenzione	1	da 16 a 128
strati impilati	1	da 40 a oltre 100
pesi in tutto	una trentina	centinaia di miliardi
testo di addestramento	3 frasi	migliaia di miliardi di token
operazioni usate	moltiplica, somma, soglia	le stesse

E il ciclo completo, quando scrivi a un modello, è questo: la tua frase viene spezzata in token, ogni token diventa un vettore, si somma la posizione, il tutto attraversa cento volte la coppia attenzione più feed-forward, la softmax dà una probabilità a ogni parola del vocabolario, se ne estrae una, la si riattacca in fondo alla frase e **si ricomincia da capo**. Una parola alla volta, senza alcun piano su come finirà la frase.

Il fatto che da questo procedimento esca qualcosa che sembra ragionare è, ancora oggi, la cosa che i ricercatori spiegano peggio.

24. Esercizi

1. Con i pesi finali $w = (0,5; -0,2)$ e $b = -0,3$, calcola z e $\hat{y}$ per un sabato con 2 amici e 4 chilometri. Poi con 5 amici e 10 chilometri.
2. Scrivi la disequazione $z \geq 0$ con quei pesi e risolvi la rispetto a x_2 . Quanti chilometri al massimo si è disposti a fare con 3 amici? E con 4?
3. Disegna sul quadretto la retta $x_2 = 2,5x_1 - 1,5$ trovando due suoi punti. Poi verifica che il punto $(2, 5)$ ci stia sopra e il punto $(2, 3)$ ci stia sotto, prima guardando il disegno e poi calcolando il segno di z .
4. Trova i valori di w_1 , w_2 e b che danno una retta con pendenza 2 e intercetta 0, cioè esattamente la regola vera “ogni amico vale due chilometri”. Attenzione: le soluzioni sono infinite. Perché?
5. Rifai l'epoca 1 partendo da pesi tutti a zero ($w = (0; 0)$, $b = 0$), sempre con $\eta = 0,1$. Arrivi agli stessi pesi finali? Arrivi comunque a zero errori?
6. Che cosa succede alla retta se moltiplichiamo w_1 , w_2 e b tutti e tre per 10? E le risposte del perceptron cambiano? Spiega perché.
7. Dimostra, con lo stesso ragionamento usato per lo XOR, che un singolo perceptron *riesce* a fare la funzione AND. Ti basta trovare tre numeri che funzionino e verificare le quattro righe.
8. Nell'esempio di backpropagation, rifai il passo di aggiornamento usando $\eta = 1$ invece di 2. L'errore scende ancora? Di più o di meno?
9. Nella backpropagation abbiamo trovato $\delta_a = -1/32$ partendo da $\delta_{\text{out}} = -1/8$. Se ci fosse un terzo strato nascosto, quanto varrebbe il δ successivo, nell'ipotesi migliore? E dopo cinque strati?
10. Perché la ReLU non soffre del gradiente che svanisce? Che cosa succede invece ai neuroni la cui uscita è sempre negativa?

Le otto cose da ricordare

1. Un perceptron fa sempre e solo tre cose: moltiplica per i pesi, somma con il bias, decide.
2. Il bias è la soglia travestita, ed è quello che permette alla frontiera di non passare per l'origine.
3. La frontiera è una retta perché $w_1x_1 + w_2x_2 + b = 0$ è un'equazione di primo grado. La pendenza è $-w_1/w_2$, l'intercetta è $-b/w_2$, e il segno di w_2 dice da che parte sta il sì.
4. I pesi ruotano la retta, il bias la trasla. Durante l'addestramento la retta si muove, e non sempre nella direzione giusta al primo colpo.
5. Uno strato solo disegna una retta, quindi ci sono problemi (lo XOR) che non potrà mai risolvere.
6. Uno strato nascosto non traccia curve: sposta i punti in un altro spazio, dove una retta basta.
7. La backpropagation è la regola della catena applicata a una rete: i “cambi” si moltiplicano lungo il percorso all'indietro.
8. Il deep learning non è “più strati”: è più strati *più* i trucchi per farli imparare (ReLU, inizializzazione, connessioni residue), *più* le architetture giuste, *più* i dati e le schede grafiche.

Per chi vuole andare a leggere gli originali. Rosenblatt, *The Perceptron*, 1958. Minsky e Papert, *Perceptrons*, 1969. Rumelhart, Hinton e Williams, *Learning representations by back-propagating errors*, *Nature*, 1986. Bengio, Simard e Frasconi, *Learning long-term dependencies with gradient descent is difficult*, 1994. Bengio e altri, *A Neural Probabilistic Language Model*, 2003. Krizhevsky, Sutskever e Hinton, *ImageNet Classification with Deep CNNs*, 2012. Bahdanau, Cho e Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, 2014. Vaswani e altri, *Attention Is All You Need*, 2017.

ESERCIZIO 11 · per chi ha voglia di rischiare

Rosenblatt ha inventato il perceptron per imitare il neurone umano. Non poteva saperlo, ma quella intuizione ha cambiato la storia.

Oggi la potenza di una rete è più o meno proporzionale a quanti neuroni riesci a farci stare dentro senza far esplodere le schede grafiche. Tutto il progresso degli ultimi anni è consistito nel farcene stare di più.

Adesso tocca a te, e non con la matematica ma con l'immaginazione.

Descrivi, solo a livello di idea, un modello che potrebbe superare le reti neurali.

Non deve imitare il cervello umano. Non deve imitare nessuna forma di intelligenza esistente. Non deve assomigliare a niente di quello che hai letto in questa dispensa. Deve solo essere, per come lo immagini tu:

parallelizzabile, semplice, veloce, efficiente.

Non ti serve saperlo programmare e non ti serve dimostrare che funziona. Ti serve solo spiegare, in mezza pagina, quale operazione elementare sta al posto della somma pesata, come farebbe a imparare dagli errori, e perché dovrebbe costare meno.

Usa la fantasia. Puoi inventare qualcosa che non assomiglia a nulla di esistente. Le idee sbagliate, in questo esercizio, valgono quanto quelle giuste: nel 1958 nessuno sapeva che moltiplicare e sommare dei numeri avrebbe portato fin qui.

25. Un tentativo di risposta all'ultimo esercizio

Da leggere solo dopo aver provato. Quello che segue è un tentativo di rispondere all'esercizio 11. Non è una proposta seria di ricerca: non l'ho mai addestrata, non ne conosco le prestazioni e più avanti troverai un elenco onesto dei motivi per cui potrebbe non funzionare. Serve a mostrarti che si può ragionare su un'idea nuova con gli stessi strumenti che hai usato in questa dispensa, e a darti un bersaglio da criticare.

25.1 La domanda da cui sono partito

Guardando all'indietro tutta questa dispensa si nota una cosa curiosa. Dal 1958 a oggi è cambiato quasi tutto:

- la funzione di attivazione: gradino, poi sigmoide, poi ReLU;
- il modo di correggere: regola locale, poi discesa del gradiente, poi backpropagation;
- l'architettura: uno strato, poi molti, poi convoluzioni, poi attenzione.

Ma una cosa non è mai cambiata, in settant'anni: **l'operazione elementare è sempre stata la somma pesata**. Moltiplica ogni numero per un peso e somma. Tutte le rivoluzioni hanno riguardato *come regolare* quei pesi, mai il fatto che si trattasse di moltiplicazioni e somme.

E quella scelta si porta dietro tutti i guai che hai incontrato. I gradienti svaniscono perché ogni strato *moltiplica* il segnale per un numero minore di uno. I modelli costano perché uno strato denso richiede n^2 moltiplicazioni. Per allenarli bisogna tenere in memoria tutte le attivazioni, perché le somme non sono invertibili: da 7 non risali a $3 + 4$.

Allora la domanda che mi sono posto è: *e se l'operazione elementare non fosse la somma, ma il confronto?*

25.2 Il mattone: un pin

Prendi due numeri, a e b , e un solo parametro imparabile, una soglia θ . La regola è:

se $a - b + \theta \geq 0$ allora **scambia**, altrimenti **lascia stare**.

Chiamo questo aggeggio un **pin**, come i chiodi di un flipper.

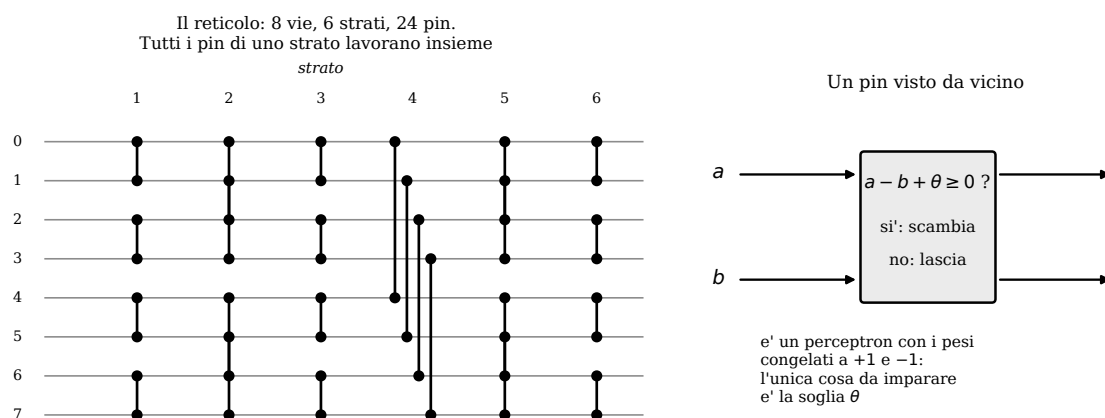


Figura 30: A sinistra il reticolo completo su 8 vie: 6 strati, 24 pin. I pin di uno stesso strato lavorano su coppie diverse, quindi tutti insieme. A destra il singolo pin.

A MANO Guarda bene la condizione $a - b + \theta \geq 0$ e confrontala con la prima pagina di questa dispensa. È

$$w_1 x_1 + w_2 x_2 + b \geq 0 \quad \text{con} \quad w_1 = +1, \quad w_2 = -1, \quad b = \theta$$

cioè è un perceptron con i pesi congelati a $+1$ e -1 , in cui l'unica cosa da imparare è il bias. Non ho inventato un mattone nuovo: ho preso quello di Rosenblatt e gli ho tolto quasi tutto. Il punto interessante è cosa succede quando di questi mattoni ne metti tanti, e soprattutto cosa si fa del loro risultato.

25.3 Il reticolo, e dove finisce la risposta

I pin si dispongono a reticolo, come i chiodi di un flipper o come le celle di un setaccio: da qui il nome che gli ho dato, **il Setaccio**. Lo schema di cablaggio non è inventato: è quello delle reti di ordinamento di Batcher, che esistono dal 1968 e hanno profondità $\log^2 n$ invece di n .

E qui viene la parte diversa. In una rete neurale la risposta è il *valore* che esce dall'ultimo neurone. Nel Setaccio i valori non vengono mai modificati: un pin li può solo scambiare di posto. Quindi la risposta non può stare in un valore.

La risposta sta nella disposizione.

Si infila fra i dati un valore speciale, un *segnalino*. Dopo il reticolo si guarda in quale casella è finito: quel numero di casella è la risposta.

È l'idea dell'abaco. Le palline non cambiano mai, cambia solo dove stanno, e la posizione è il numero.

A MANO Proviamo un reticolo minuscolo, 4 vie e 5 pin, con tutte le soglie a zero. I pin sono, nell'ordine: (0, 1) e (2, 3) nel primo strato, (0, 2) e (1, 3) nel secondo, (1, 2) nel terzo. Ingresso [3, 7, 1, 9].

strato	pin	il conto	risultato
1	(0, 1): $a = 3, b = 7$	$3 - 7 + 0 = -4 < 0$	lascia
1	(2, 3): $a = 1, b = 9$	$1 - 9 + 0 = -8 < 0$	lascia
dopo lo strato 1:			[3, 7, 1, 9]
2	(0, 2): $a = 3, b = 1$	$3 - 1 + 0 = +2 \geq 0$	scambia
2	(1, 3): $a = 7, b = 9$	$7 - 9 + 0 = -2 < 0$	lascia
dopo lo strato 2:			[1, 7, 3, 9]
3	(1, 2): $a = 7, b = 3$	$7 - 3 + 0 = +4 \geq 0$	scambia
dopo lo strato 3:			[1, 3, 7, 9]

Con tutte le soglie a zero il reticolo ordina. Ma le soglie non sono a zero: sono imparate, e ogni pin ha la sua. Un θ diverso da zero significa “scambia solo se a supera b di almeno tanto”, cioè introduce un handicap personalizzato in quel punto preciso del reticolo. Con soglie tutte diverse il reticolo non ordina più: fa qualcos'altro, e quel qualcos'altro è la funzione imparata.

25.4 Come impara

E qui torna il problema che attraversa tutta questa dispensa: se la risposta è sbagliata, *di chi è la colpa?*

Rosenblatt	la colpa è degli input accesi. Ecco perché la correzione è proporzionale a x_i .
backpropagation	la colpa si divide con la regola della catena, e per farlo bisogna toccare <i>tutti</i> i pesi.
il Setaccio	la colpa è dei pin che il segnalino ha effettivamente toccato lungo la sua strada. Sono $\log^2 n$, non tutti.

Il segnalino, attraversando il reticolo, incontra esattamente un pin per strato. Se è finito troppo a sinistra, tutti i pin del suo percorso avevano la soglia un filo troppo bassa; se troppo a destra, un filo troppo alta. La correzione è di una semplicità imbarazzante e ricalca quella di Rosenblatt:

$$\theta \leftarrow \theta \pm 1 \quad \text{solo per i pin toccati}$$

A MANO Vediamola su un pin solo. Compito: rispondere 1 se a supera b di almeno 3. Soglia iniziale $\theta = 0$.

a	b	y	θ	$z = a - b + \theta$	$\hat{y}$	nuovo θ
5	1	1	0	+4	1	giusto
3	2	0	0	+1	1	-1
7	2	1	-1	+4	1	giusto
4	4	0	-1	-1	0	giusto
6	1	1	-1	+4	1	giusto
2	5	0	-1	-4	0	giusto
5	3	0	-1	+1	1	-2
8	1	1	-2	+5	1	giusto

Fine della prima epoca: 2 errori, $\theta = -2$. Nella seconda epoca sbaglia solo la riga (5, 3) e θ scende a -3. Nella terza epoca non sbaglia più niente: con $\theta = -3$ la condizione $a - b - 3 \geq 0$ significa esattamente $a \geq b + 3$, che era la regola da imparare.

È la stessa identica tabella dell'epoca 1 della sezione 7, con la stessa forma e lo stesso lieto fine. Solo che qui il numero da imparare è uno solo, ed è intero.

25.5 Le tre cose che lo rendono interessante

Prima: il gradiente non può svanire

Questa è la proprietà che mi convince di più, perché risolve di netto il problema della sezione 18.

Il gradiente svanisce perché ogni strato *moltiplica* il segnale per un fattore minore di uno: con la sigmoide, al massimo 0,25, e $0,25^{20}$ è zero. Nel Setaccio non c'è nessuna moltiplicazione, quindi non c'è nessun fattore. Un valore che entra nel reticolo esce identico, solo in un'altra casella. Cento strati non lo consumano più di uno.

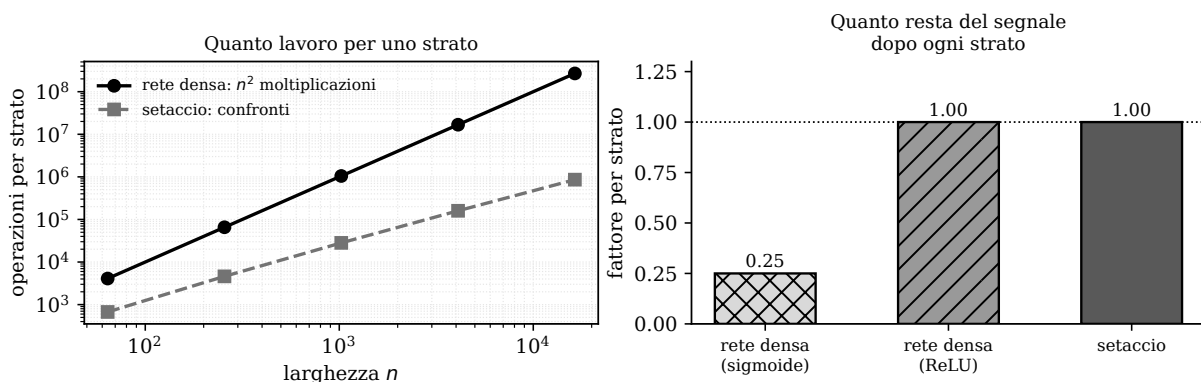


Figura 31: A sinistra il lavoro di calcolo per uno strato. A destra quanto resta del segnale dopo ogni strato: nel Setaccio è esattamente 1, perché i valori non vengono mai riscalati.

Seconda: costa poco, e nella moneta giusta

Uno strato denso su n numeri richiede n^2 moltiplicazioni. Il reticolo richiede $\frac{n}{2} \log^2 n$ confronti, e un confronto è la cosa più economica che un processore sappia fare.

n	moltiplicazioni (denso)	confronti (Setaccio)	profondità
1 024	1 048 576	28 160	55
16 384	268 435 456	860 160	105

A $n = 16\,384$ sono trecento volte meno operazioni, e per giunta più economiche una per una. In più i parametri sono piccoli interi, non numeri in virgola mobile: la memoria crolla.

Terza: si può tornare indietro senza aver ricordato niente

Questa mi sembra la conseguenza più elegante, e non l'avevo prevista quando ho cominciato a ragionarci.

Per addestrare una rete normale bisogna conservare tutte le attivazioni del passo in avanti, perché servono nel passo all'indietro. È la voce di spesa più grossa nell'addestramento dei modelli grandi. La ragione è che la somma non si può invertire: sapendo che il risultato è 7 non sai se veniva da $3 + 4$ o da $5 + 2$.

Uno scambio invece si inverte facendolo di nuovo. Basta annotare, per ogni pin, **un bit**: ho scambiato oppure no. Con quei bit si ripercorre il reticolo all'indietro e si ricostruisce esattamente l'ingresso.

A MANO Nel reticolo a 4 vie di prima i bit erano $[0, 0, 1, 0, 1]$. Parti da $[1, 3, 7, 9]$, rifai i pin in ordine inverso scambiando dove il bit vale 1, e ottieni di nuovo $[3, 7, 1, 9]$. Provaci.

Cinque bit al posto di dodici numeri in virgola mobile, in questo esempio minuscolo. Su un modello vero il rapporto sarebbe di uno a centinaia.

25.6 Perché potrebbe non funzionare

Adesso la parte che in una proposta di ricerca vale più di tutte le altre.

1. **Il Setaccio non può creare niente di nuovo.** I numeri che escono sono esattamente quelli che sono entrati, in ordine diverso. Non può fare medie, non può sommare due informazioni in una terza. Questa è una limitazione seria e non un dettaglio: significa che il Setaccio non è un rimpiazzo delle reti neurali, ma un tipo di modello diverso, buono per problemi diversi.
2. **Calcola funzioni d'ordine, non funzioni lineari.** Sa rispondere a domande del tipo “chi è il più grande”, “quanti ce ne sono sopra la soglia”, “in che ordine stanno”. Su un problema che è genuinamente una somma pesata, come il nostro cinema della prima pagina, va male.
3. **Ha poca capacità per strato.** L'informazione che una disposizione di n elementi può contenere è $\log_2(n!)$ bit: per $n = 1024$ sono circa 8 800 bit, cioè un kilobyte. Uno strato denso della stessa larghezza ha un milione di pesi. Servirebbero moltissimi reticoli in parallelo per compensare, e a quel punto il vantaggio sui costi si riduce.
4. **La regola locale è un'approssimazione, e potrebbe non bastare.** Dire che la colpa è solo dei pin toccati dal segnalino è comodo ma non è del tutto vero: anche i pin *non* toccati hanno deciso quali numeri il segnalino avrebbe incontrato. È esattamente il problema dell'attribuzione della responsabilità che ha fermato le reti per diciassette anni, e qui si ripresenta travestito. Potrebbe essere che una regola locale sia sufficiente in pratica, come lo era per Rosenblatt su problemi separabili; potrebbe anche essere che l'addestramento si inchiodi su reticoli profondi. Non lo so, e senza provare non lo sa nessuno.
5. **I parametri sono interi, quindi il paesaggio dell'errore è a scalini.** Non c'è una discesa liscia da seguire: si procede a salti, e i salti possono girare in tondo.

25.7 Che cosa esiste già, per onestà

Nessuna idea nasce dal nulla, e sarebbe scorretto lasciarti credere che questa sia piovuta dal cielo. Ecco i parenti stretti che conosco:

reti di ordinamento (Batcher, 1968)	il cablaggio del reticolo è preso di lì paro paro, e la profondità $\log^2 n$ è un loro risultato
reti di permutazione di Beneš	instradare segnali con interruttori invece che con l'aritmetica è un'idea vecchia delle telecomunicazioni
macchine di Tsetlin, reti senza pesi	modelli che imparano regole discrete invece di numeri reali
ordinamento derivabile, max-pooling	pezzi di reti moderne che sono già, di fatto, operazioni d'ordine

Quello che non mi risulta sia mai stato messo insieme è la combinazione: un reticolo di comparatori con soglia appresa, addestrato con una regola locale lungo il percorso, in cui la risposta si legge dalla posizione di un segnalino invece che da un valore. Se qualcuno di voi trova che esiste già, quella è una scoperta più utile dell'idea stessa: significa che potete andare a leggere come è finita.

25.8 La scommessa

C'è un argomento a favore che vale la pena di lasciarti, perché è il tipo di ragionamento che fa nascere le idee.

Guarda che cosa fanno le reti moderne nei loro momenti decisivi. La softmax serve per lo più a far emergere il valore più grande. L'attenzione assegna pesi in base a chi somiglia di più, cioè fa una classifica. La previsione finale di un LLM è sostanzialmente un ordinamento del vocabolario, di cui poi si guarda la cima.

*E se buona parte di quello che una rete calcola faticosamente
con miliardi di moltiplicazioni fosse, alla fine, informazione d'ordine?*

Se fosse vero, calcolarla direttamente nel mondo degli ordinamenti invece di ricavarla dall'aritmetica sarebbe una scorciatoia enorme. Se fosse falso, questa idea non serve a niente.

Non so quale delle due. Ma è una domanda a cui si può rispondere con un esperimento, e le domande di questo tipo sono le uniche che facciano avanzare qualcosa.

Che cosa portarti via da questo capitolo

Non l'idea del Setaccio, che quasi certamente ha qualche difetto che non ho visto. Piuttosto il metodo, che è sempre lo stesso:

1. cerca l'ipotesi che nessuno mette in discussione perché è lì da troppo tempo (qui: che calcolare significhi moltiplicare e sommare);

2. cambia quella, e una sola;

3. guarda che cosa cade e che cosa resta in piedi;

4. elenca onestamente i motivi per cui la tua idea dovrebbe fallire, prima che lo faccia qualcun altro.

È esattamente quello che ha fatto Rosenblatt nel 1958, e la cosa più bella è che non serviva sapere in anticipo che avrebbe funzionato.